People's Democratic Republic of Algeria

Ministry of Higher Education and Scientific Research

Mohamed Seddik Ben Yahia University - Jijel

Faculty of Sciences and Technology

Department of Computer Science

School Management System

UML Analysis and Design Report

Specialty: Computer Science (Informatics)

Level: 3rd Year License

Academic Session: 2025 - 2026

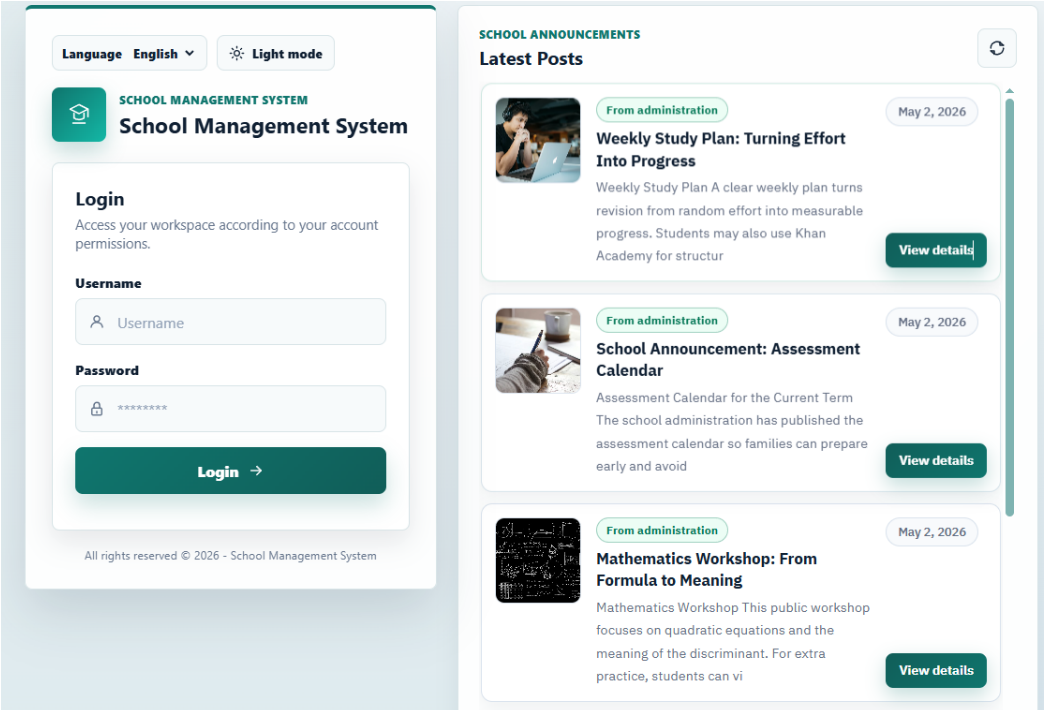
Prepared by

1. Bouremouz Mouhammed
2. Debieche Amine
3. Bessibes Mouin
4. Chaabani Sidahmed
5. Bouzekria Sohib
6. Meriche Chemseddine

Under the supervision of

Dr. El Hillali Kerkouche

Executive Summary



Login interface preview

The School Management System is a complete web application for organizing school administration, academic follow-up, communication, and finance. It is built around a frontend/backend architecture and six role-based dashboards: Admin, Receptionist, Accountant, Teacher, Student, and Parent.

Project goal	Digitize daily school workflows, reduce manual follow-up, and give each actor a secure workspace.
Architecture	Static HTML/CSS/JavaScript frontend connected to a Python FastAPI backend and relational database.
Frontend structure	Shared core modules manage API calls, authentication, routing, preferences, storage, and i18n; each role has its own page, services, controller, and UI renderer.
Backend structure	FastAPI routers and database managers cover users, parents, students, teachers, classes, enrollments, assignments, schedules, attendance, assessments, grades, resources, fees, payments, transactions, messages, notifications, posts, and programs.
Data design	The initial structure was studied from an Excel data reference from Al Abakera School, then transformed into relational entities and UML models.
Deployment	The project was prepared for hosted demonstration using frontend/backend port forwarding and cloud services for remote access.
Documentation	The UML report includes use case diagrams, class diagram, sequence diagrams, navigation diagram, technology notes, GitHub repository, and tooling notes.

Role Coverage

Admin	Manages users, parents, students, teachers, classes, programs, enrollments, attendance, schedules, assessments, grades, resources, finance, posts, notifications, and conversations.
Receptionist	Handles student and parent registration, student files, search, finance lookup, payments, posts, messages, and notifications.
Accountant	Follows student fees, payments, transactions, financial files, attendance reports, messages, notifications, and finance notices.

Teacher	Uses assignments, schedules, attendance sheets, assessments, grade entry, educational resources, posts, messages, and notifications.
Student	Consults schedule, assessments, grades, attendance, resources, fees, posts, messages, and notifications.
Parent	Follows linked children, grades, attendance, fees, posts, messages, and notifications.

In short, this summary gives the reader the project scope, technical structure, main features, data-modeling approach, deployment context, and UML documentation strategy before reading the detailed diagrams and sections.

Table of Contents

Introduction 5

Project Context and Benefits 5

Functional Models: Use Case Diagrams 5

Admin Use Case Diagram 7

Receptionist Use Case Diagram 8

Accountant Use Case Diagram 9

Teacher Use Case Diagram 10

Student Use Case Diagram 11

Parent Use Case Diagram 12

Structural Model: Class Diagram 13

School Management System Class Diagram 14

Dynamic Models: Sequence Diagrams 15

Admin Sequence Diagram 16

Receptionist Sequence Diagram 17

Accountant Sequence Diagram 18

Teacher Sequence Diagram 19

Student Sequence Diagram 20

Parent Sequence Diagram 21

User Interface: Navigation Diagram 22

Role-Based Navigation Diagram 23

Technology Stack, Deployment, and Repository 24

Application Interface Showcase 25

Conclusion 30

Introduction

This report presents the analysis and design of a web-based School Management System. The application is organized around six role-based dashboards: Admin, Receptionist, Accountant, Teacher, Student, and Parent.

The documentation includes a concise project context, use case diagrams, a class diagram, sequence diagrams, a navigation diagram, technology/deployment notes, and a conclusion.

Project Context and Benefits

Opening Statement

The project is a role-based web application for school management. It connects administration, reception, accounting, teachers, students, and parents through one frontend/backend platform.

System Benefits

Benefit	Contribution to the School Management System
Administrative efficiency	Centralizes records, enrollments, schedules, and follow-up.
Pedagogical follow-up	Tracks assessments, grades, attendance, and resources.
Communication	Uses messages, notifications, and administration posts.
Financial clarity	Supports fees, payments, transactions, and reminders.
Role-based organization	Separates dashboards by responsibility and user role.

Initial Data Modeling with Al Abakera Excel Data

At the beginning, an Excel data reference from Al Abakera School was used to identify the main entities: users, students, parents, teachers, classes, enrollments, fees, payments, attendance, schedules, assessments, grades, and resources.

Hosting, Port Forwarding, and Cloud Services

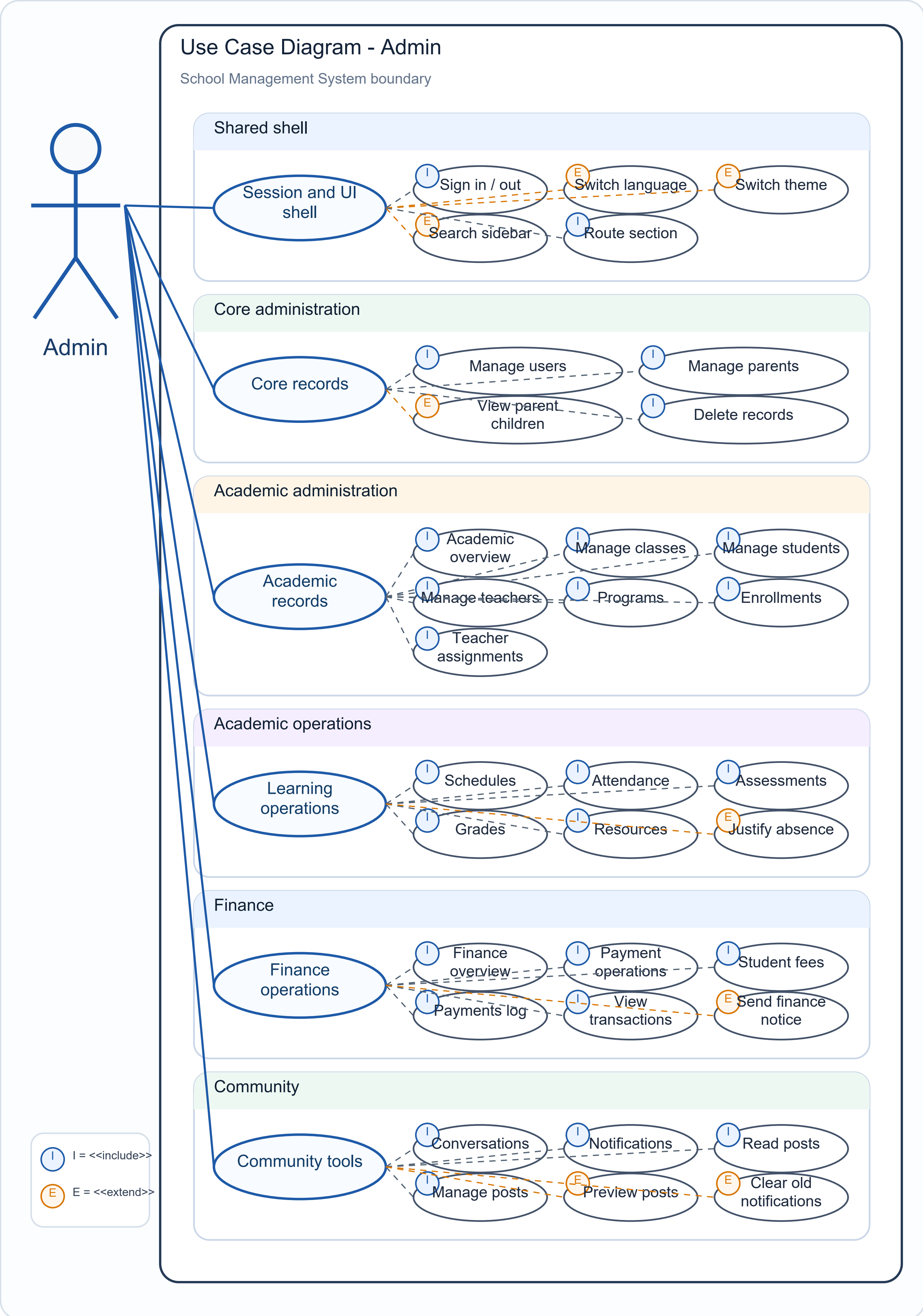
Port forwarding was configured for both frontend and backend services to support remote testing. Cloud services were used to make the hosted demonstration accessible outside the local network.

Functional Models: Use Case Diagrams

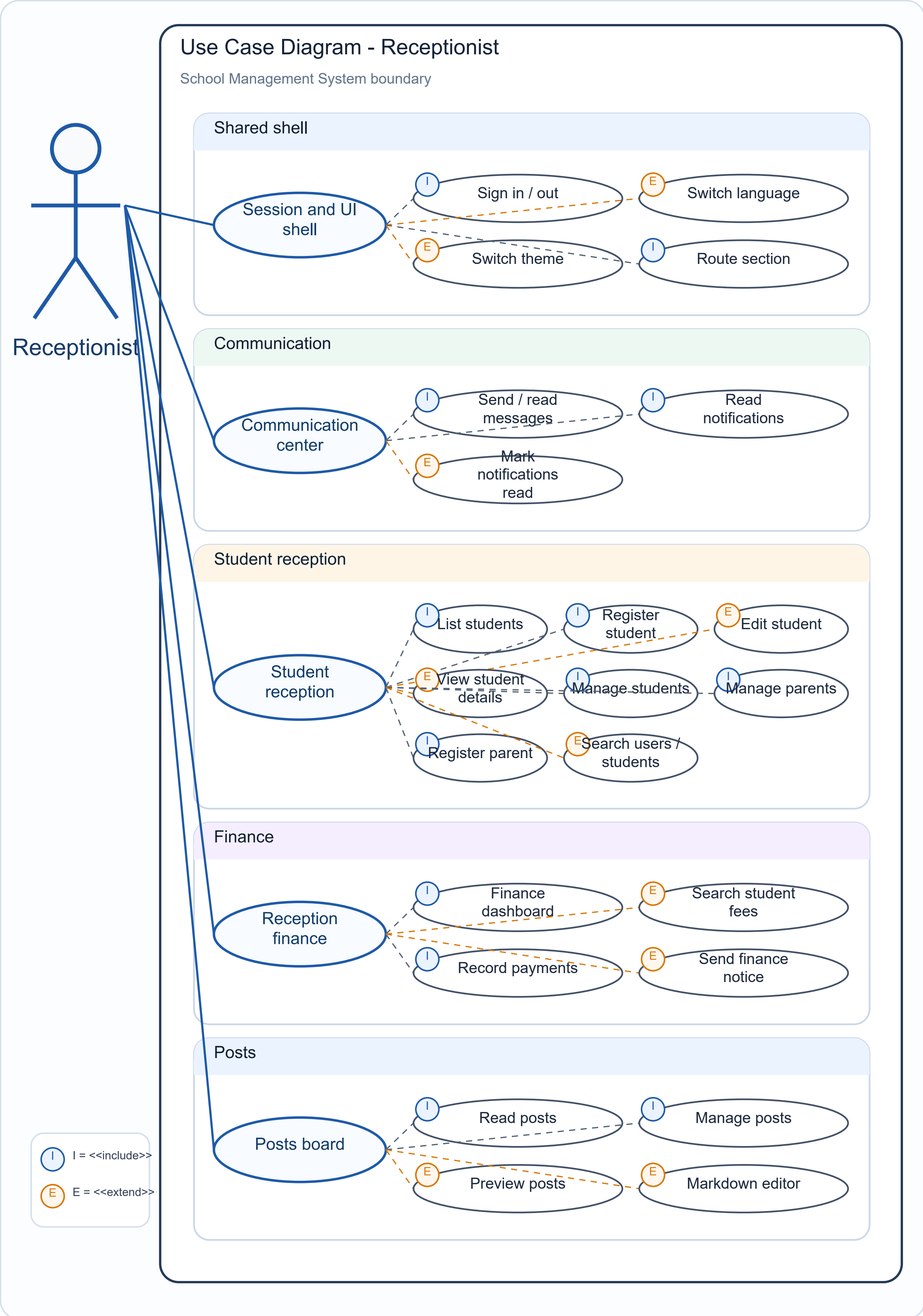
The use case diagrams define the visible services of the School Management System for each role. They show the functional boundary of every dashboard and clarify which operations belong to administration, reception, accounting, teaching, learner self-service, and parent follow-up.

Use Case Scope

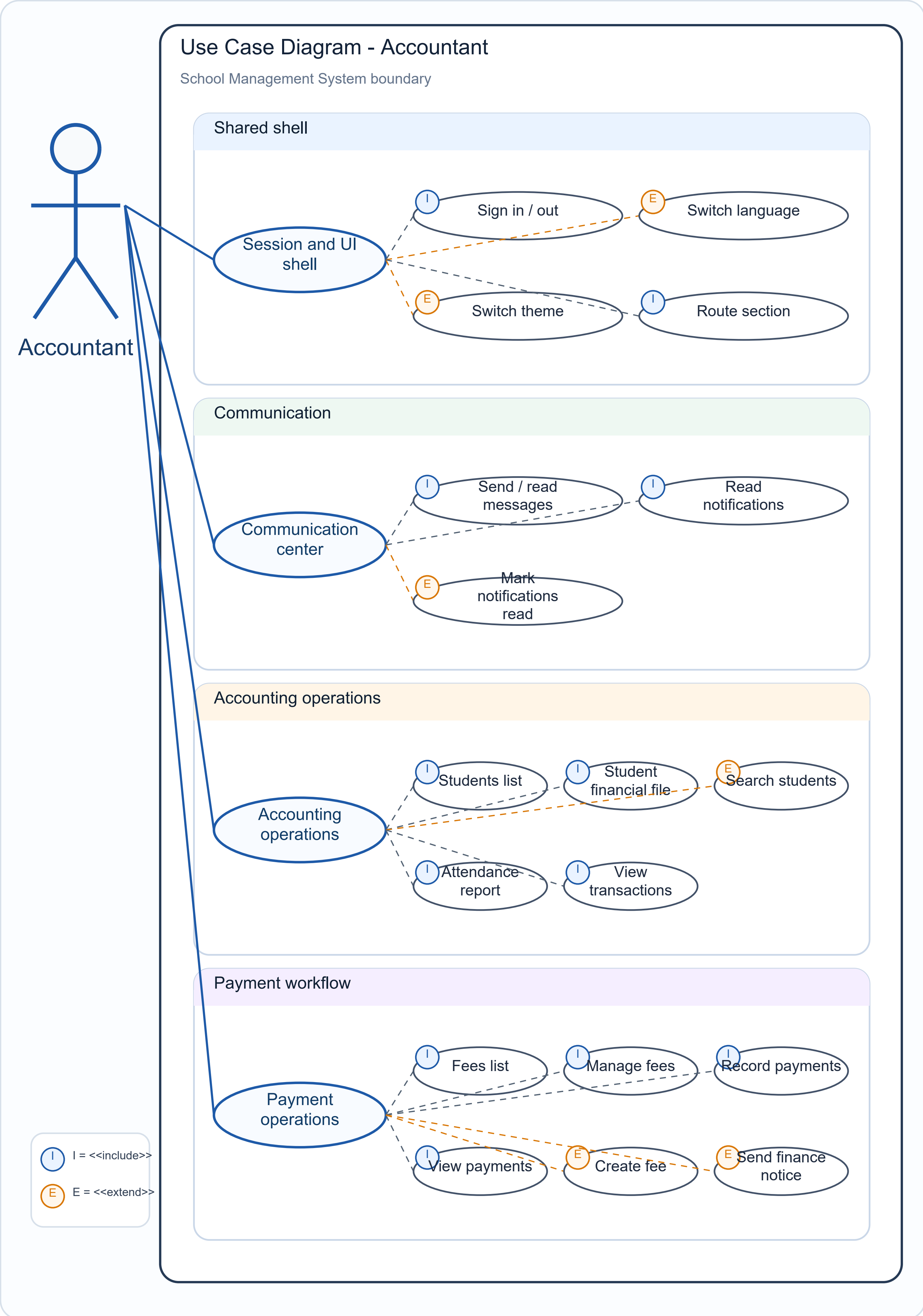
Admin: complete system administration across people, academic, finance, posts, and notifications.
Receptionist: front-desk workflows for students, parents, records, finance lookup, posts, and communication.
Accountant: fees, payments, transactions, student financial files, notices, and attendance reports.
Teacher: assignments, schedules, attendance, assessments, grades, resources, messages, and notifications.
Student: read-only academic self-service: schedule, assessments, grades, attendance, resources, posts, fees, messages, and notifications.
Parent: child monitoring: grades, attendance, fees, posts, messages, and notifications.

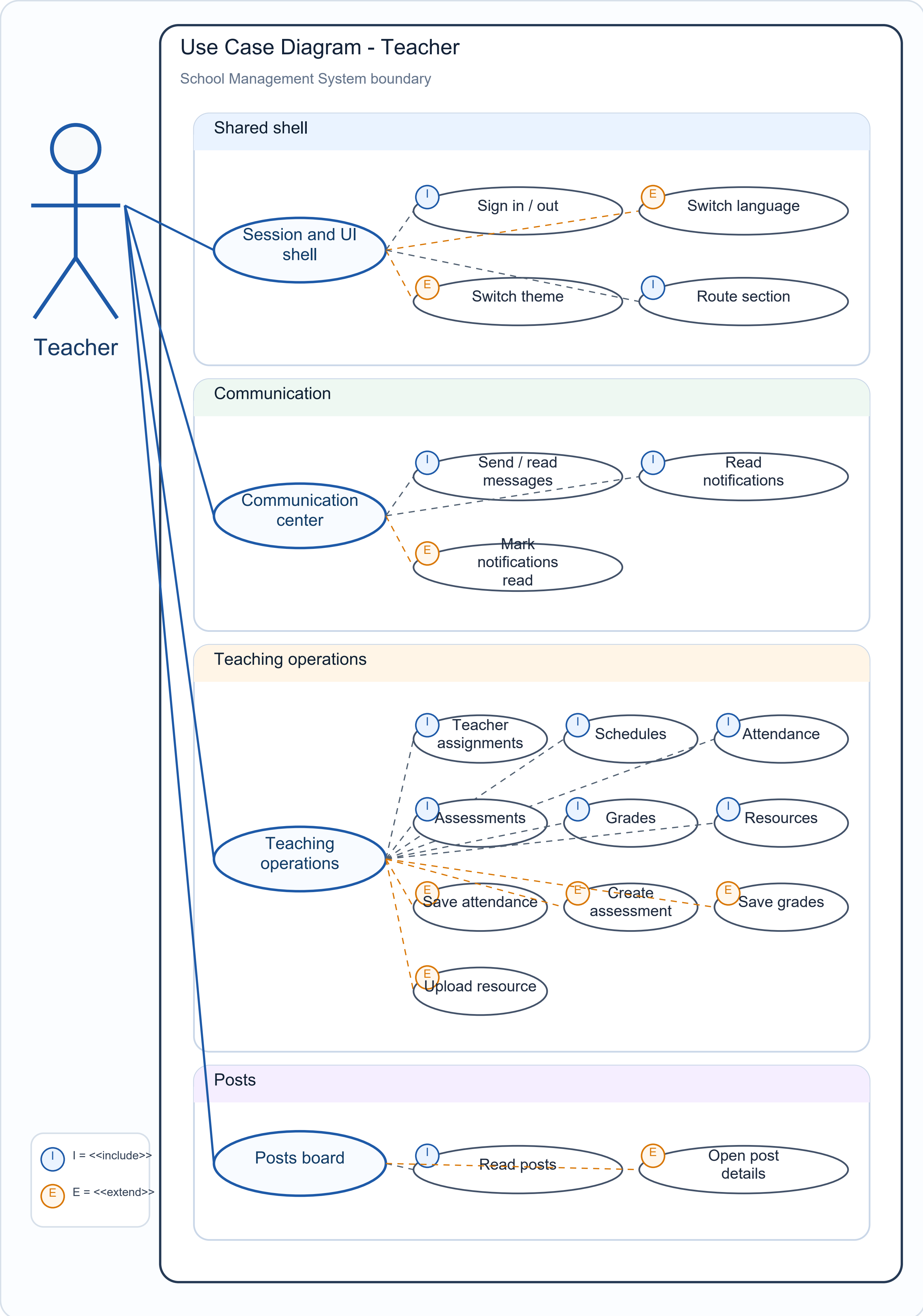


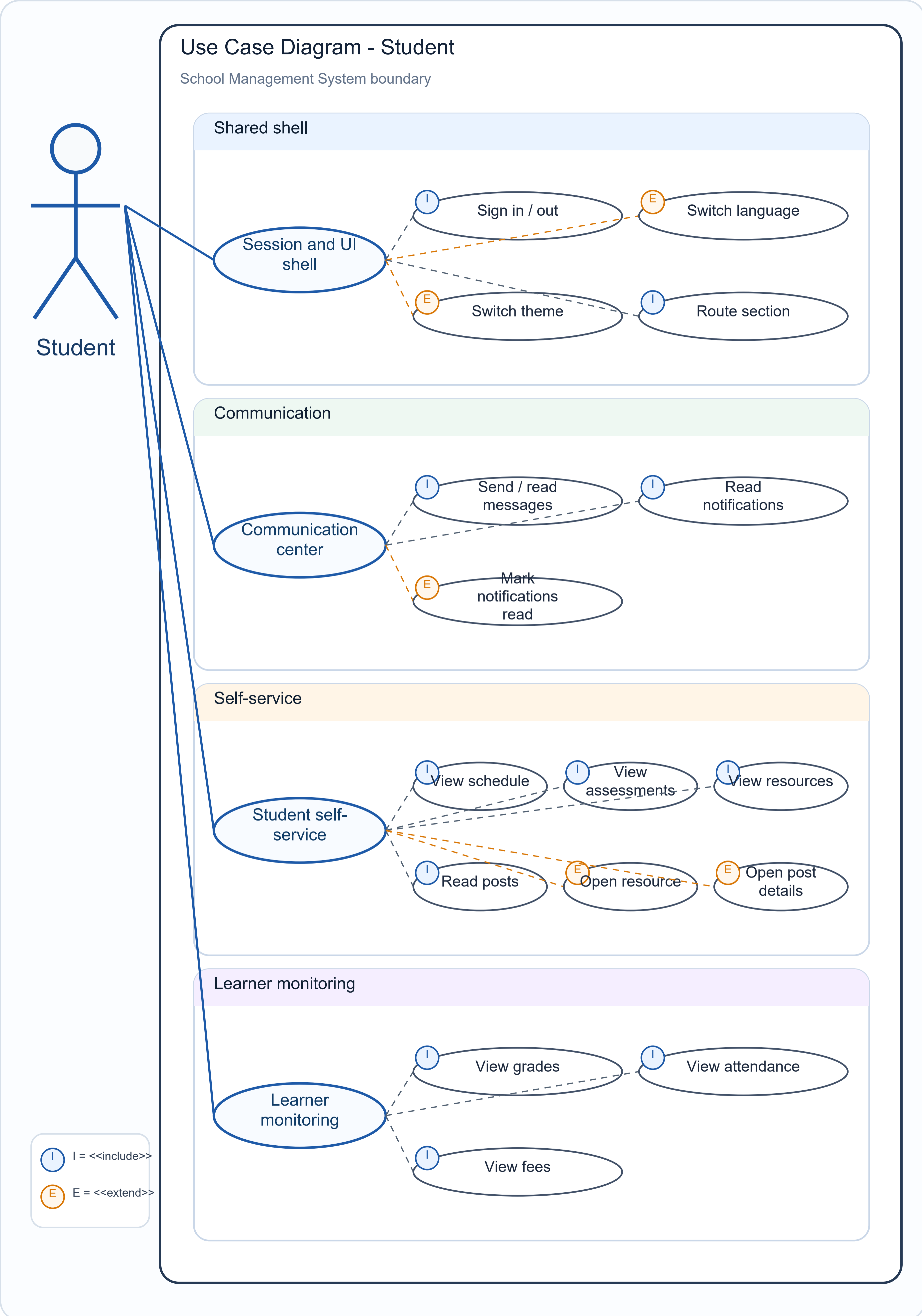
Use Case Diagram - Admin
School Management System boundary
Admin
Shared shell
Session and UI shell
Sign in / out
Switch language
Switch theme
Search sidebar
Route section
Core administration
Core records
Manage users
Manage parents
View parent children
Delete records
Academic administration
Academic records
Academic overview
Manage classes
Manage students
Manage teachers
Programs
Enrollments
Teacher assignments
Academic operations
Learning operations
Schedules
Attendance
Assessments
Grades
Resources
Justify absence
Finance
Finance operations
Finance overview
Payment operations
Student fees
Payments log
View transactions
Send finance notice
Community
Community tools
Conversations
Notifications
Read posts
Manage posts
Preview posts
Clear old notifications
I = <<include>>
E = <<extend>>

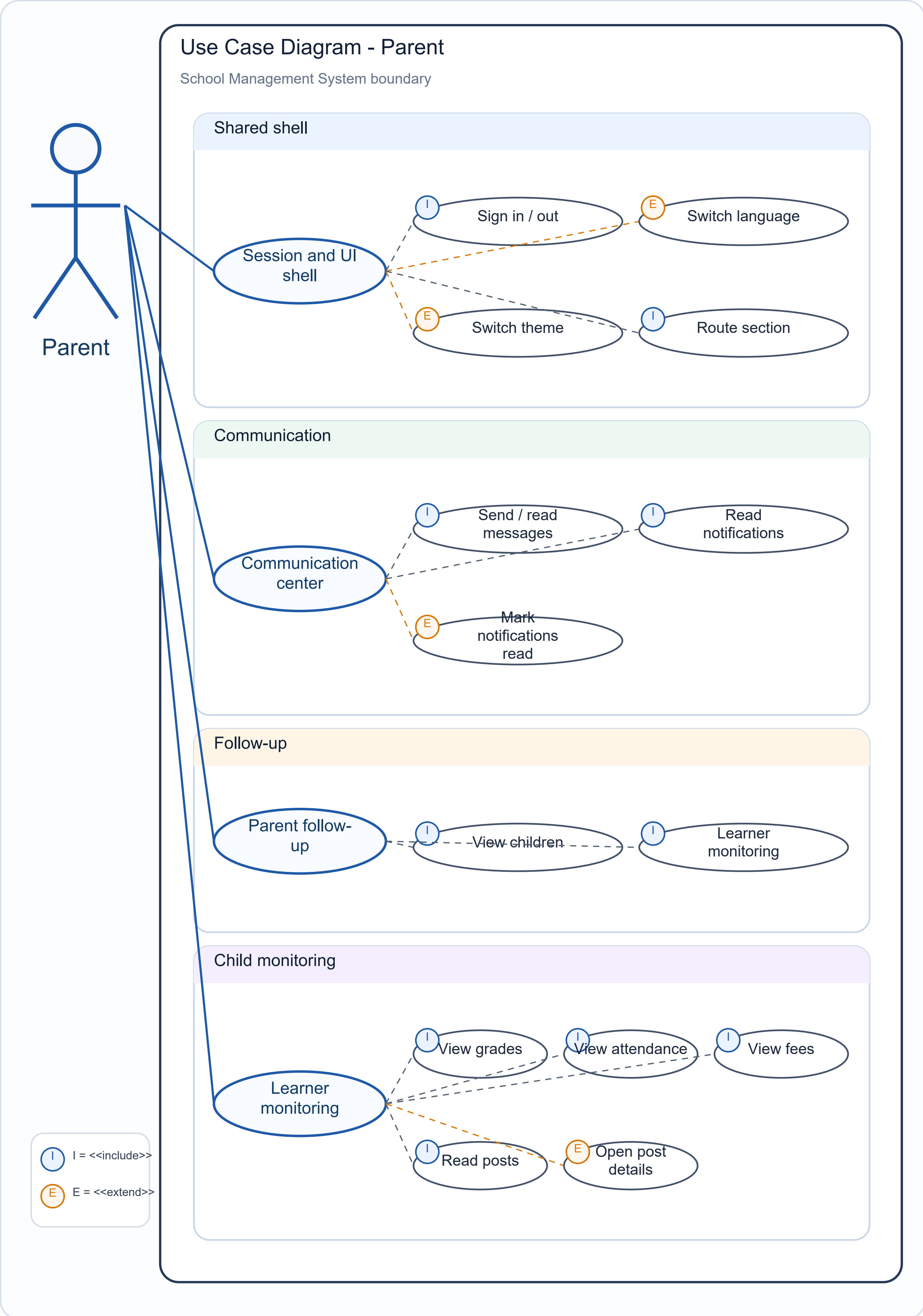


Use Case Diagram - Receptionist
School Management System boundary
Receptionist
Shared shell
Session and UI shell
Sign in / out
Switch language
Switch theme
Route section
Communication
Communication center
Send / read messages
Read notifications
Mark notifications read
Student reception
Student reception
List students
Register student
Edit student
View student details
Manage students
Manage parents
Register parent
Search users / students
Finance
Reception finance
Finance dashboard
Search student fees
Record payments
Send finance notice
Posts
Posts board
Read posts
Manage posts
Preview posts
Markdown editor
I = <<include>>
E = <<extend>>


Use Case Diagram - Accountant
School Management System boundary
Accountant
Shared shell
Session and UI shell
Sign in / out
Switch language
Switch theme
Route section
Communication
Communication center
Send / read messages
Read notifications
Mark notifications read
Accounting operations
Accounting operations
Students list
Student financial file
Search students
Attendance report
View transactions
Payment workflow
Payment operations
Fees list
Manage fees
Record payments
View payments
Create fee
Send finance notice
I = <<include>>
E = <<extend>>


Use Case Diagram - Teacher
School Management System boundary
Teacher
Shared shell
Session and UI shell
Sign in / out
Switch language
Switch theme
Route section
Communication
Communication center
Send / read messages
Read notifications
Mark notifications read
Teaching operations
Teaching operations
Teacher assignments
Schedules
Attendance
Assessments
Grades
Resources
Save attendance
Create assessment
Save grades
Upload resource
Posts
Posts board
Read posts
Open post details
I = <<include>>
E = <<extend>>


Use Case Diagram - Student
School Management System boundary
Student
Shared shell
Session and UI shell
Sign in / out
Switch language
Switch theme
Route section
Communication
Communication center
Send / read messages
Read notifications
Mark notifications read
Self-service
Student self-service
View schedule
View assessments
View resources
Read posts
Open resource
Open post details
Learner monitoring
Learner monitoring
View grades
View attendance
View fees
I = <<include>>
E = <<extend>>



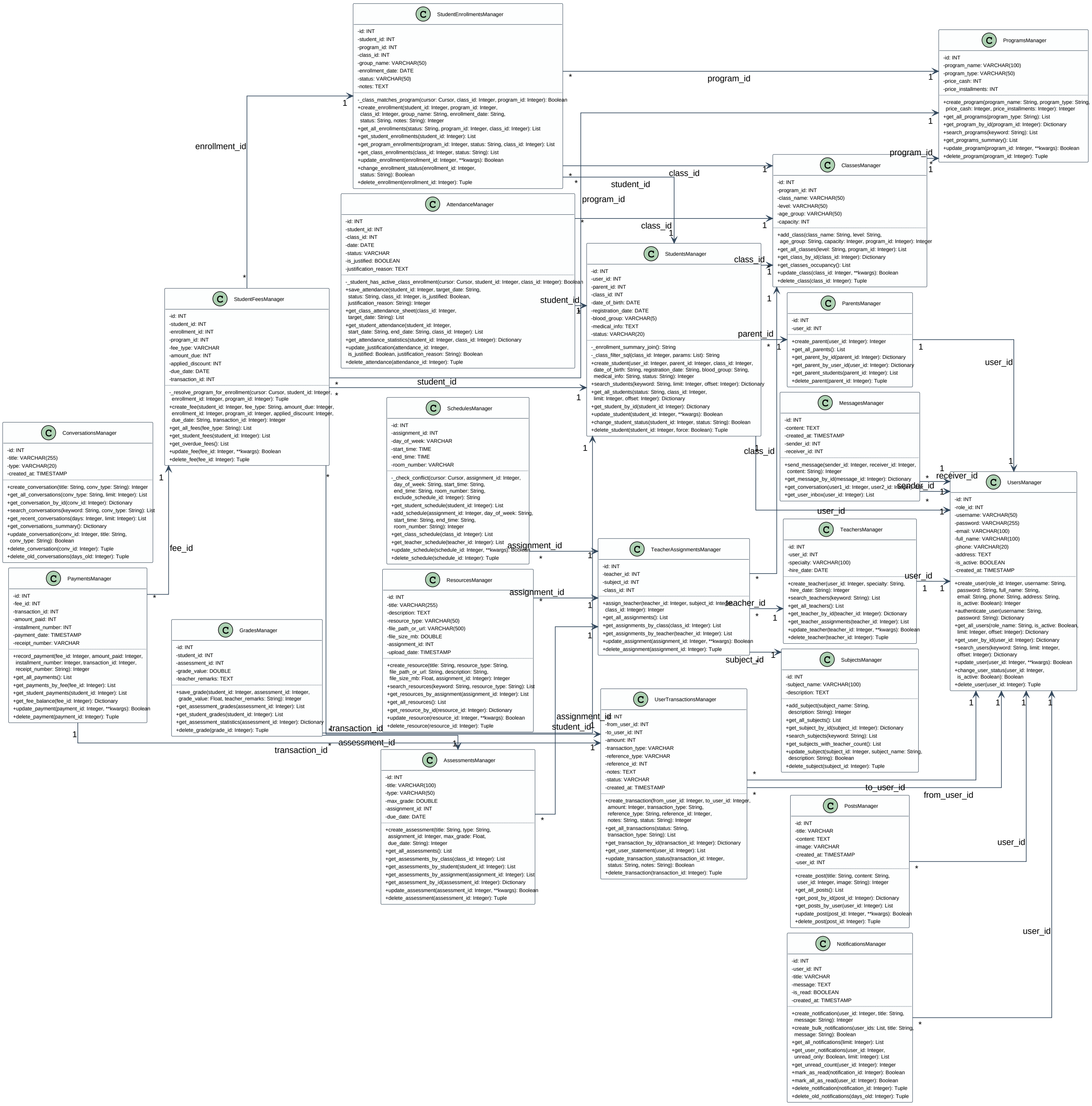
Use Case Diagram - Parent
School Management System boundary
Parent
Shared shell
Session and UI shell
Sign in / out
Switch language
Switch theme
Route section
Communication
Communication center
Send / read messages
Read notifications
Mark notifications read
Follow-up
Parent follow-up
View children
Learner monitoring
Child monitoring
Learner monitoring
View grades
View attendance
View fees
Read posts
Open post details
I = <<include>>
E = <<extend>>
I
E

Structural Model: Class Diagram

The class diagram presents the backend structure of the School Management System. It groups the database managers and domain entities used to support identity, academic management, learning operations, finance, communication, and publication features.

Structural Responsibilities

Identity and people: users, parents, students, teachers, authentication, and role ownership.
Academic structure: programs, classes, enrollments, subjects, and teacher assignments.
Learning operations: schedules, attendance, assessments, grades, and educational resources.
Finance: student fees, payments, and user transactions.
Communication: conversations, messages, notifications, and posts.


StudentEnrollmentsManager
-id: INT
-student_id: INT
-program_id: INT
-class_id: INT
-group_name: VARCHAR(50)
-enrollment_date: DATE
-status: VARCHAR(50)
-notes: TEXT
-_class_matches_program(cursor: Cursor, class_id: Integer, program_id: Integer): Boolean
+create_enrollment(student_id: Integer, program_id: Integer, class_id: Integer, group_name: String, enrollment_date: String, status: String, notes: String): Integer
+get_all_enrollments(status: String, program_id: Integer, class_id: Integer): List
+get_student_enrollments(student_id: Integer): List
+get_program_enrollments(program_id: Integer, status: String, class_id: Integer): List
+get_class_enrollments(class_id: Integer, status: String): List
+update_enrollment(enrollment_id: Integer, **kwargs): Boolean
+change_enrollment_status(enrollment_id: Integer, status: String): Boolean
+delete_enrollment(enrollment_id: Integer): Tuple
ProgramsManager
-id: INT
-program_name: VARCHAR(100)
-program_type: VARCHAR(50)
-price_cash: INT
-price_installments: INT
+create_program(program_name: String, program_type: String, price_cash: Integer, price_installments: Integer): Integer
+get_all_programs(program_type: String): List
+get_program_by_id(program_id: Integer): Dictionary
+search_programs(keyword: String): List
+get_programs_summary(): List
+update_program(program_id: Integer, **kwargs): Boolean
+delete_program(program_id: Integer): Tuple
AttendanceManager
-id: INT
-student_id: INT
-class_id: INT
-date: DATE
-status: VARCHAR
-is_justified: BOOLEAN
-justification_reason: TEXT
-_student_has_active_class_enrollment(cursor: Cursor, student_id: Integer, class_id: Integer): Boolean
+save_attendance(student_id: Integer, target_date: String, status: String, class_id: Integer, is_justified: Boolean, justification_reason: String): Integer
+get_class_attendance_sheet(class_id: Integer, target_date: String): List
+get_student_attendance(student_id: Integer, start_date: String, end_date: String, class_id: Integer): List
+get_attendance_statistics(student_id: Integer, class_id: Integer): Dictionary
+update_justification(attendance_id: Integer, is_justified: Boolean, justification_reason: String): Boolean
+delete_attendance(attendance_id: Integer): Tuple
ClassesManager
-id: INT
-program_id: INT
-class_name: VARCHAR(50)
-level: VARCHAR(50)
-age_group: VARCHAR(50)
-capacity: INT
+add_class(class_name: String, level: String, age_group: String, capacity: Integer, program_id: Integer): Integer
+get_all_classes(level: String, program_id: Integer): List
+get_class_by_id(class_id: Integer): Dictionary
+get_classes_occupancy(): List
+update_class(class_id: Integer, **kwargs): Boolean
+delete_class(class_id: Integer): Tuple
StudentsManager
-id: INT
-user_id: INT
-parent_id: INT
-class_id: INT
-date_of_birth: DATE
-registration_date: DATE
-blood_group: VARCHAR(5)
-medical_info: TEXT
-status: VARCHAR(20)
-_enrollment_summary_join(): String
-_class_filter_sql(class_id: Integer, params: List): String
+create_student(user_id: Integer, parent_id: Integer, class_id: Integer, date_of_birth: String, registration_date: String, blood_group: String, medical_info: String, status: String): Integer
+search_students(keyword: String, limit: Integer, offset: Integer): Dictionary
+get_all_students(status: String, class_id: Integer, limit: Integer, offset: Integer): Dictionary
+get_student_by_id(student_id: Integer): Dictionary
+update_student(student_id: Integer, **kwargs): Boolean
+change_student_status(student_id: Integer, status: String): Boolean
+delete_student(student_id: Integer, force: Boolean): Tuple
StudentFeesManager
-id: INT
-student_id: INT
-enrollment_id: INT
-program_id: INT
-fee_type: VARCHAR
-amount_due: INT
-applied_discount: INT
-due_date: DATE
-transaction_id: INT
-_resolve_program_for_enrollment(cursor: Cursor, student_id: Integer, enrollment_id: Integer, program_id: Integer): Tuple
+create_fee(student_id: Integer, fee_type: String, amount_due: Integer, enrollment_id: Integer, program_id: Integer, applied_discount: Integer, due_date: String, transaction_id: Integer): Integer
+get_all_fees(fee_type: String): List
+get_student_fees(student_id: Integer): List
+get_overdue_fees(): List
+update_fee(fee_id: Integer, **kwargs): Boolean
+delete_fee(fee_id: Integer): Tuple
ParentsManager
-id: INT
-user_id: INT
+create_parent(user_id: Integer): Integer
+get_all_parents(): List
+get_parent_by_id(parent_id: Integer): Dictionary
+get_parent_by_user_id(user_id: Integer): Dictionary
+get_parent_students(parent_id: Integer): List
+delete_parent(parent_id: Integer): Tuple
MessagesManager
-id: INT
-content: TEXT
-created_at: TIMESTAMP
-sender_id: INT
-receiver_id: INT
+send_message(sender_id: Integer, receiver_id: Integer, content: String): Integer
+get_message_by_id(message_id: Integer): Dictionary
+get_conversation(user1_id: Integer, user2_id: Integer): List
+get_user_inbox(user_id: Integer): List
ConversationsManager
-id: INT
-title: VARCHAR(255)
-type: VARCHAR(20)
-created_at: TIMESTAMP
+create_conversation(title: String, conv_type: String): Integer
+get_all_conversations(conv_type: String, limit: Integer): List
+get_conversation_by_id(conv_id: Integer): Dictionary
+search_conversations(keyword: String, conv_type: String): List
+get_recent_conversations(days: Integer, limit: Integer): List
+get_conversations_summary(): Dictionary
+update_conversation(conv_id: Integer, title: String, conv_type: String): Boolean
+delete_conversation(conv_id: Integer): Tuple
+delete_old_conversations(days_old: Integer): Tuple
SchedulesManager
-id: INT
-assignment_id: INT
-day_of_week: VARCHAR
-start_time: TIME
-end_time: TIME
-room_number: VARCHAR
-_check_conflict(cursor: Cursor, assignment_id: Integer, day_of_week: String, start_time: String, end_time: String, room_number: String, exclude_schedule_id: Integer): String
+get_student_schedule(student_id: Integer): List
+add_schedule(assignment_id: Integer, day_of_week: String, start_time: String, end_time: String, room_number: String): Integer
+get_class_schedule(class_id: Integer): List
+get_teacher_schedule(teacher_id: Integer): List
+update_schedule(schedule_id: Integer, **kwargs): Boolean
+delete_schedule(schedule_id: Integer): Tuple
UsersManager
-id: INT
-role_id: INT
-username: VARCHAR(50)
-password: VARCHAR(255)
-email: VARCHAR(100)
-full_name: VARCHAR(100)
-phone: VARCHAR(20)
-address: TEXT
-is_active: BOOLEAN
-created_at: TIMESTAMP
+create_user(role_id: Integer, username: String, password: String, full_name: String, email: String, phone: String, address: String, is_active: Boolean): Integer
+authenticate_user(username: String, password: String): Dictionary
+get_all_users(role_name: String, is_active: Boolean, limit: Integer, offset: Integer): Dictionary
+get_user_by_id(user_id: Integer): Dictionary
+search_users(keyword: String, limit: Integer, offset: Integer): Dictionary
+update_user(user_id: Integer, **kwargs): Boolean
+change_user_status(user_id: Integer, is_active: Boolean): Boolean
+delete_user(user_id: Integer): Tuple
TeacherAssignmentsManager
-id: INT
-teacher_id: INT
-subject_id: INT
-class_id: INT
+assign_teacher(teacher_id: Integer, subject_id: Integer, class_id: Integer): Integer
+get_all_assignments(): List
+get_assignments_by_class(class_id: Integer): List
+get_assignments_by_teacher(teacher_id: Integer): List
+update_assignment(assignment_id: Integer, **kwargs): Boolean
+delete_assignment(assignment_id: Integer): Tuple
TeachersManager
-id: INT
-user_id: INT
-specialty: VARCHAR(100)
-hire_date: DATE
+create_teacher(user_id: Integer, specialty: String, hire_date: String): Integer
+search_teachers(keyword: String): List
+get_all_teachers(): List
+get_teacher_by_id(teacher_id: Integer): Dictionary
+get_teacher_assignments(teacher_id: Integer): List
+update_teacher(teacher_id: Integer, **kwargs): Boolean
+delete_teacher(teacher_id: Integer): Tuple
PaymentsManager
-id: INT
-fee_id: INT
-transaction_id: INT
-amount_paid: INT
-installment_number: INT
-payment_date: TIMESTAMP
-receipt_number: VARCHAR
+record_payment(fee_id: Integer, amount_paid: Integer, installment_number: Integer, transaction_id: Integer, receipt_number: String): Integer
+get_all_payments(): List
+get_payments_by_fee(fee_id: Integer): List
+get_student_payments(student_id: Integer): List
+get_fee_balance(fee_id: Integer): Dictionary
+update_payment(payment_id: Integer, **kwargs): Boolean
+delete_payment(payment_id: Integer): Tuple
GradesManager
-id: INT
-student_id: INT
-assessment_id: INT
-grade_value: DOUBLE
-teacher_remarks: TEXT
+save_grade(student_id: Integer, assessment_id: Integer, grade_value: Float, teacher_remarks: String): Integer
+get_assessment_grades(assessment_id: Integer): List
+get_student_grades(student_id: Integer): List
+get_assessment_statistics(assessment_id: Integer): Dictionary
+delete_grade(grade_id: Integer): Tuple
ResourcesManager
-id: INT
-title: VARCHAR(255)
-description: TEXT
-resource_type: VARCHAR(50)
-file_path_or_url: VARCHAR(500)
-file_size_mb: DOUBLE
-assignment_id: INT
-upload_date: TIMESTAMP
+create_resource(title: String, resource_type: String, file_path_or_url: String, description: String, file_size_mb: Float, assignment_id: Integer): Integer
+search_resources(keyword: String, resource_type: String): List
+get_resources_by_assignment(assignment_id: Integer): List
+get_all_resources(): List
+get_resource_by_id(resource_id: Integer): Dictionary
+update_resource(resource_id: Integer, **kwargs): Boolean
+delete_resource(resource_id: Integer): Tuple
SubjectsManager
-id: INT
-subject_name: VARCHAR(100)
-description: TEXT
+add_subject(subject_name: String, description: String): Integer
+get_all_subjects(): List
+get_subject_by_id(subject_id: Integer): Dictionary
+search_subjects(keyword: String): List
+get_subjects_with_teacher_count(): List
+update_subject(subject_id: Integer, subject_name: String, description: String): Boolean
+delete_subject(subject_id: Integer): Tuple
UserTransactionsManager
-id: INT
-from_user_id: INT
-to_user_id: INT
-amount: INT
-transaction_type: VARCHAR
-reference_type: VARCHAR
-reference_id: INT
-notes: TEXT
-status: VARCHAR
-created_at: TIMESTAMP
+create_transaction(from_user_id: Integer, to_user_id: Integer, amount: Integer, transaction_type: String, reference_type: String, reference_id: Integer, notes: String, status: String): Integer
+get_all_transactions(status: String, transaction_type: String): List
+get_transaction_by_id(transaction_id: Integer): Dictionary
+get_user_statement(user_id: Integer): List
+update_transaction_status(transaction_id: Integer, status: String, notes: String): Boolean
+delete_transaction(transaction_id: Integer): Tuple
AssessmentsManager
-id: INT
-title: VARCHAR(100)
-type: VARCHAR(50)
-max_grade: DOUBLE
-assignment_id: INT
-due_date: DATE
+create_assessment(title: String, type: String, assignment_id: Integer, max_grade: Float, due_date: String): Integer
+get_all_assessments(): List
+get_assessments_by_class(class_id: Integer): List
+get_assessments_by_student(student_id: Integer): List
+get_assessments_by_assignment(assignment_id: Integer): List
+get_assessment_by_id(assessment_id: Integer): Dictionary
+update_assessment(assessment_id: Integer, **kwargs): Boolean
+delete_assessment(assessment_id: Integer): Tuple
PostsManager
-id: INT
-title: VARCHAR
-content: TEXT
-image: VARCHAR
-created_at: TIMESTAMP
-user_id: INT
+create_post(title: String, content: String, user_id: Integer, image: String): Integer
+get_all_posts(): List
+get_post_by_id(post_id: Integer): Dictionary
+get_posts_by_user(user_id: Integer): List
+update_post(post_id: Integer, **kwargs): Boolean
+delete_post(post_id: Integer): Tuple
NotificationsManager
-id: INT
-user_id: INT
-title: VARCHAR
-message: TEXT
-is_read: BOOLEAN
-created_at: TIMESTAMP
+create_notification(user_id: Integer, title: String, message: String): Integer
+create_bulk_notifications(user_ids: List, title: String, message: String): Boolean
+get_all_notifications(limit: Integer): List
+get_user_notifications(user_id: Integer, unread_only: Boolean, limit: Integer): List
+get_unread_count(user_id: Integer): Integer
+mark_as_read(notification_id: Integer): Boolean
+mark_all_as_read(user_id: Integer): Boolean
+delete_notification(notification_id: Integer): Tuple
+delete_old_notifications(days_old: Integer): Tuple
program_id
enrollment_id
class_id
student_id
parent_id
user_id
fee_id
assignment_id
teacher_id
subject_id
receiver_id
sender_id
transaction_id
assessment_id
to_user_id
from_user_id

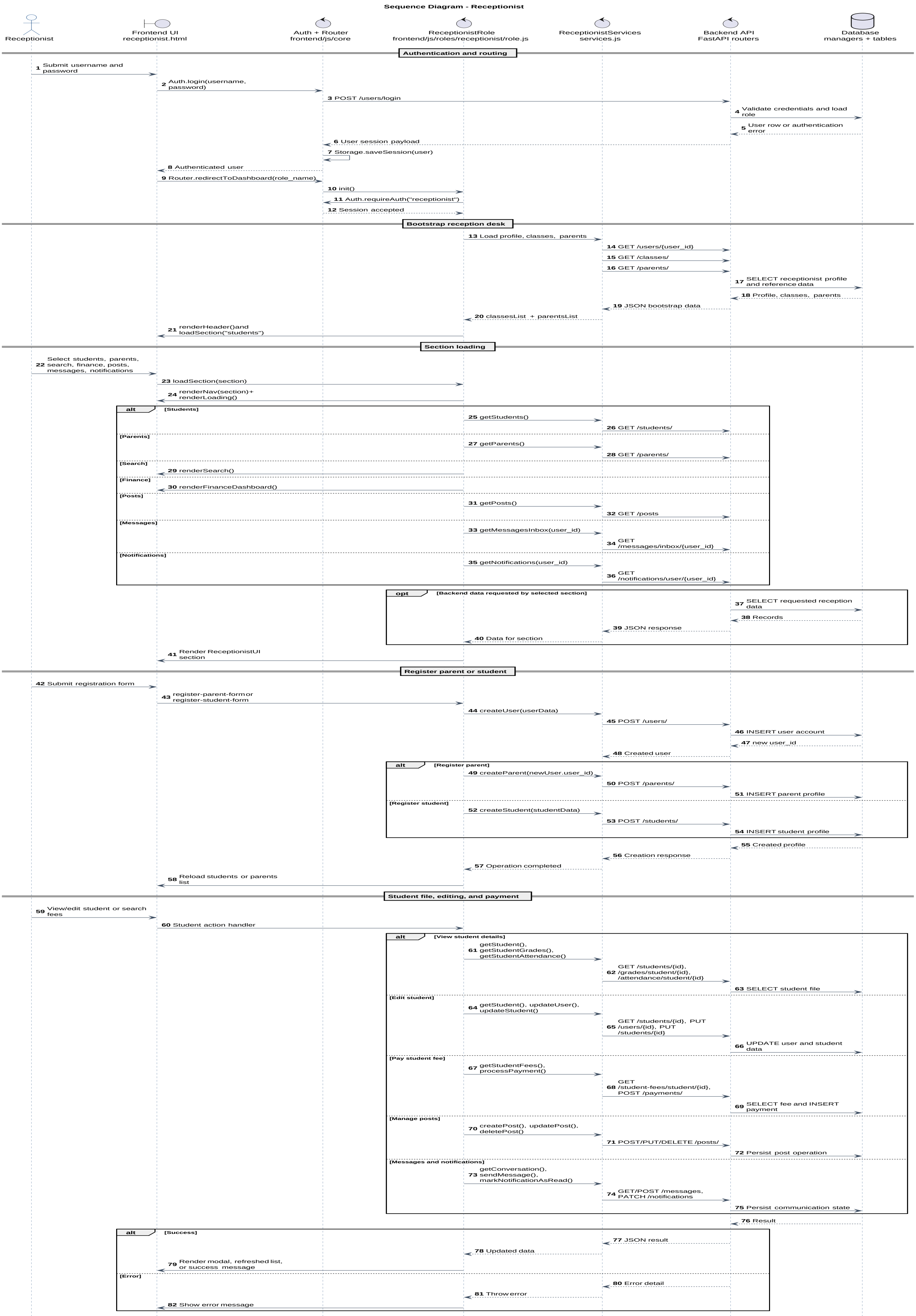
Dynamic Models: Sequence Diagrams

The sequence diagrams describe how each role interacts with the application at runtime. The common pattern is: user action in the frontend, role controller handling, service/API request, FastAPI router processing, database operation, then UI rendering of the result.

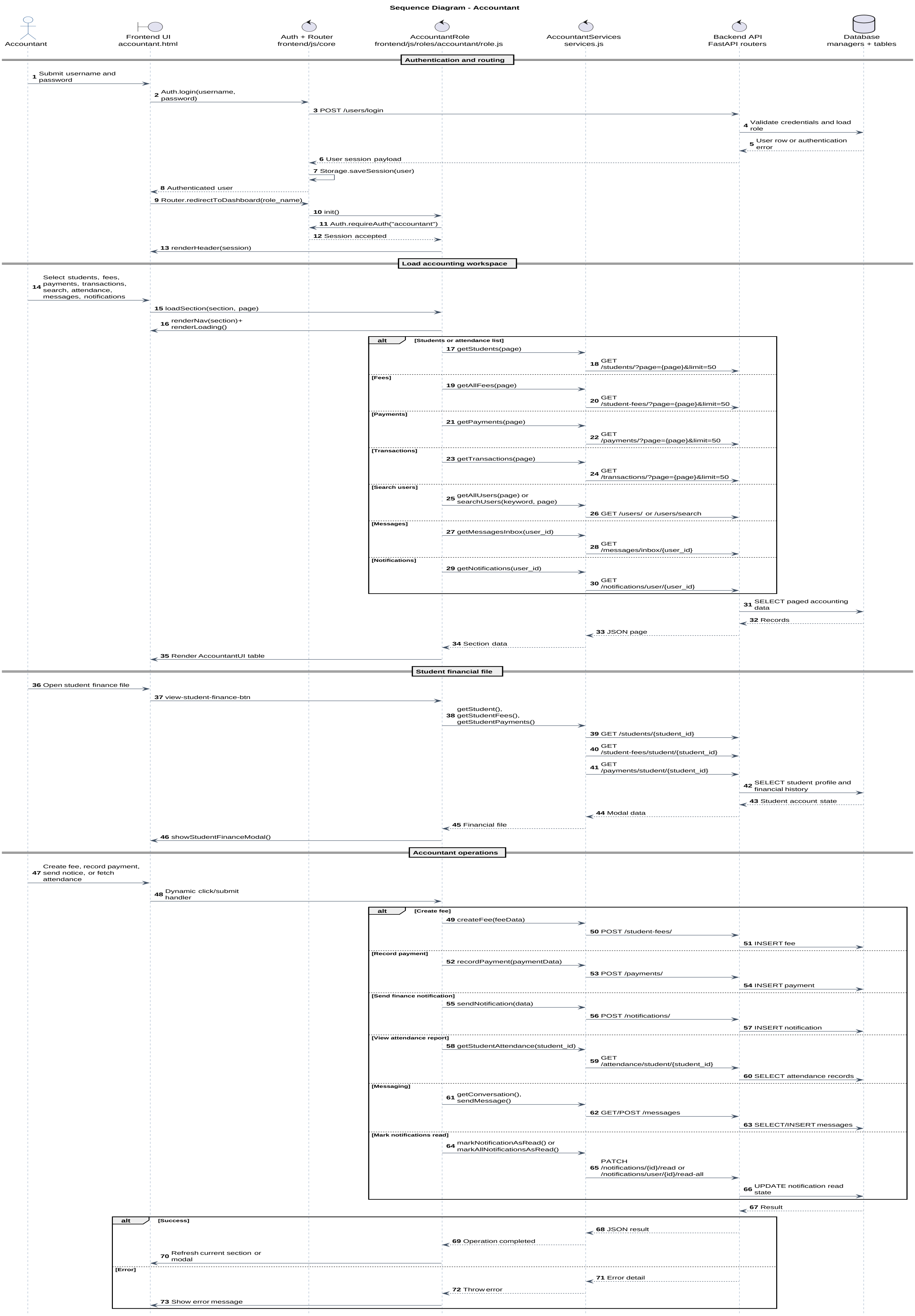
Sequence Flow Summary

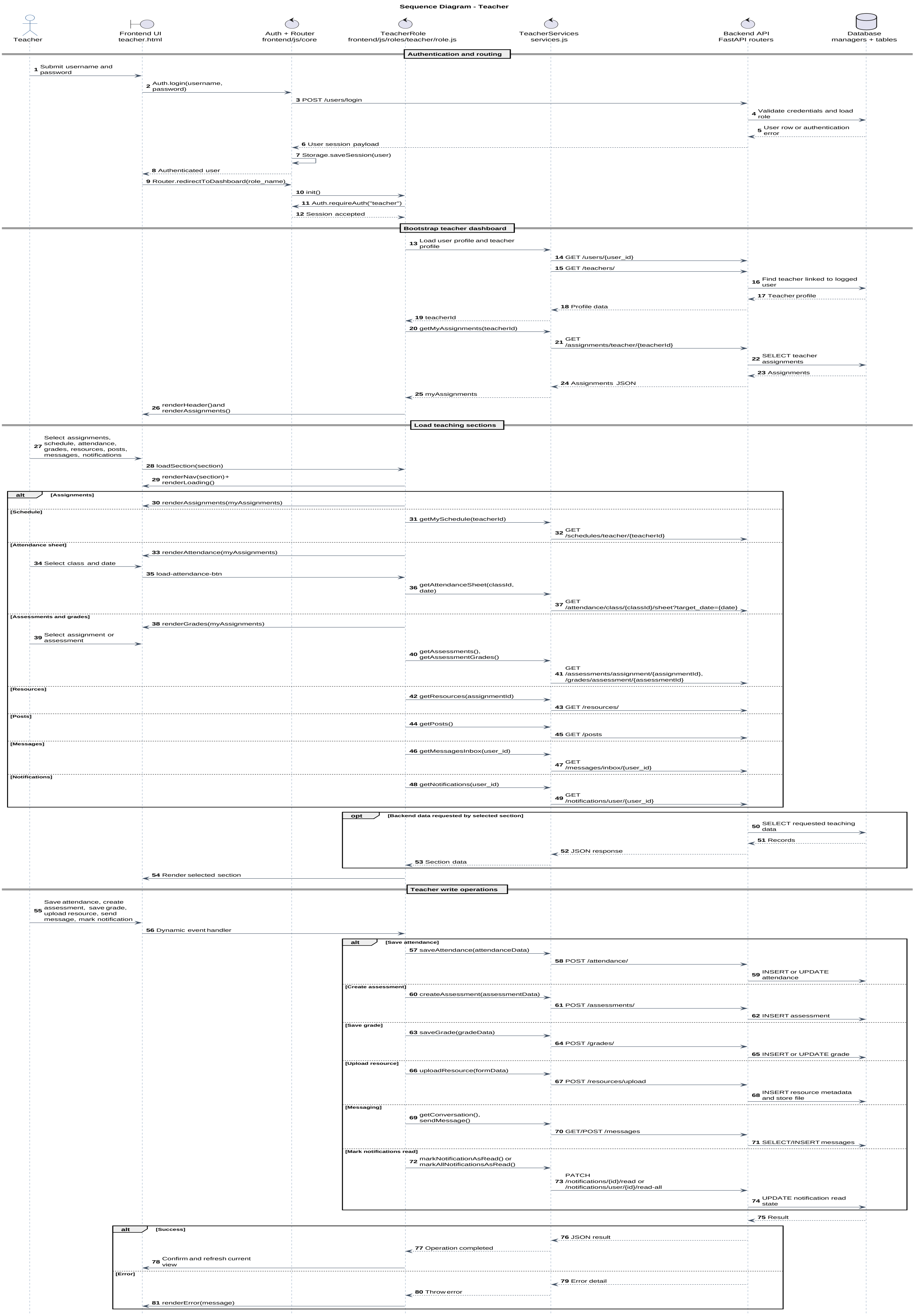
Admin: authentication, route selection, data loading, and full create/update/delete administration.
Receptionist: dashboard bootstrap, student and parent registration, student records, payments, posts, messaging, and notifications.
Accountant: finance workspace loading, student financial files, fee creation, payment recording, notices, attendance reports, messaging, and notification updates.
Teacher: teacher profile resolution, assignments, schedules, attendance, assessments, grades, resources, messaging, and notifications.
Student: linked profile resolution, academic self-service views, messages, and notifications.
Parent: parent profile resolution, linked children aggregation, child monitoring, posts, messages, and notifications.

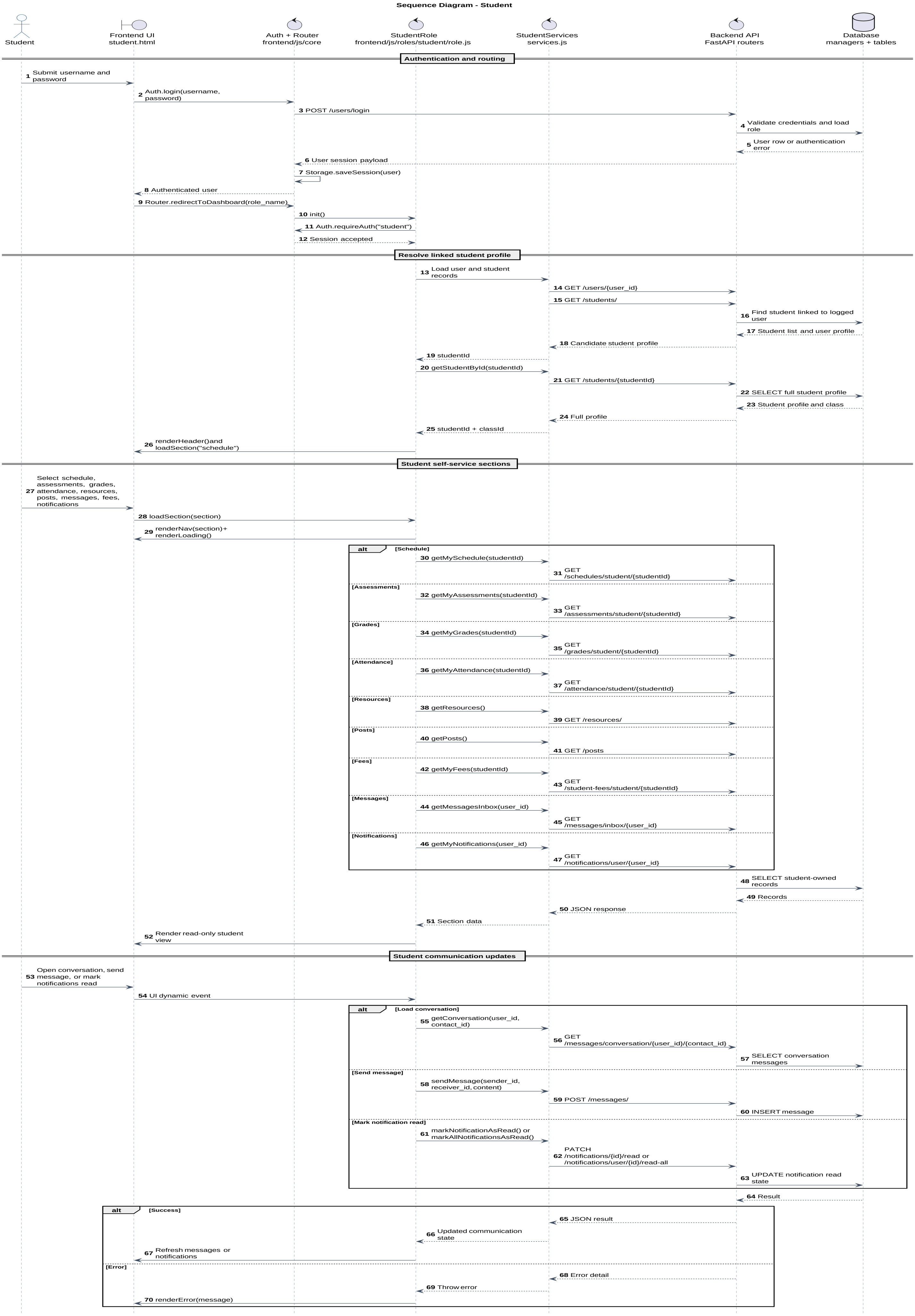

Sequence Diagram - Admin
Admin
Frontend UI admin.html
Auth + Router frontend/js/core
AdminRole frontend/js/roles/admin/role.js
Admin Services/Tabs services.js + tabs
Backend API FastAPI routers
Database managers + tables
Authentication and routing
1 Submit username and password
2 Auth.login(username, password)
3 POST /users/login
4 Validate credentials and load role
5 User row or authentication error
6 User session payload
7 Storage.saveSession(user)
8 Authenticated user
9 Router.redirectToDashboard(role_name)
10 init()
11 Auth.requireAuth("admin")
12 Session accepted
13 renderHeader(session)
14 handleRoute(hash || "academic")
Load selected administration section
15 Select sidebar section
16 loadSection(section)
17 renderNav(section)+ renderLoading()
alt [Users and people]
18 getUsers(), getParents(), getStudents(), getTeachers()
19 GET /users, /parents, /students, /teachers
[Academic structure]
20 getClasses(), getPrograms(), getEnrollments()
21 GET /classes, /programs, /enrollments
[Learning operations]
22 Load attendance, schedules, assessments, grades, resources
23 GET /attendance, /schedules, /assessments, /grades, /resources
[Finance]
24 getStudentFees(), getPayments(), getTransactions()
25 GET /student-fees, /payments, /transactions
[Community]
26 getConversations(), getNotifications(), getPosts()
27 GET /conversations, /notifications, /posts
28 SELECT requested records
29 Records
30 JSON dataset
31 Dataset for active tab
32 Render AdminUI tab
Admin write operations
33 Submit create/update/delete/status action
34 Call tab handler or deleteItem()
alt [Create or update core record]
35 POST/PUT users, parents, students, teachers, classes, programs
36 POST/PUT /users, /parents, /students, /teachers, /classes, /programs
37 INSERT or UPDATE core table
[Academic operation]
38 Manage enrollments, schedules, assessments, grades, attendance
39 POST/PUT/PATCH /enrollments, /schedules, /assessments, /grades, /attendance
40 Persist academic operation
[Finance operation]
41 Manage fees, payments, transactions, reminders
42 POST/PUT/PATCH /student-fees, /payments, /transactions, /notifications
43 Persist finance operation
[Community operation]
44 Manage posts, conversations, notifications
45 POST/PUT/PATCH/DELETE /posts, /messages, /notifications
46 Persist community operation
[Delete record]
47 deleteRecord(endpoint, id)
48 DELETE /{endpoint}/{id}
49 DELETE target record
50 Commit result
alt [Success]
51 Saved/deleted response
52 Operation completed
53 Reload active section and show success
[Validation or authorization error]
54 Error detail
55 Throw error
56 renderError(message)



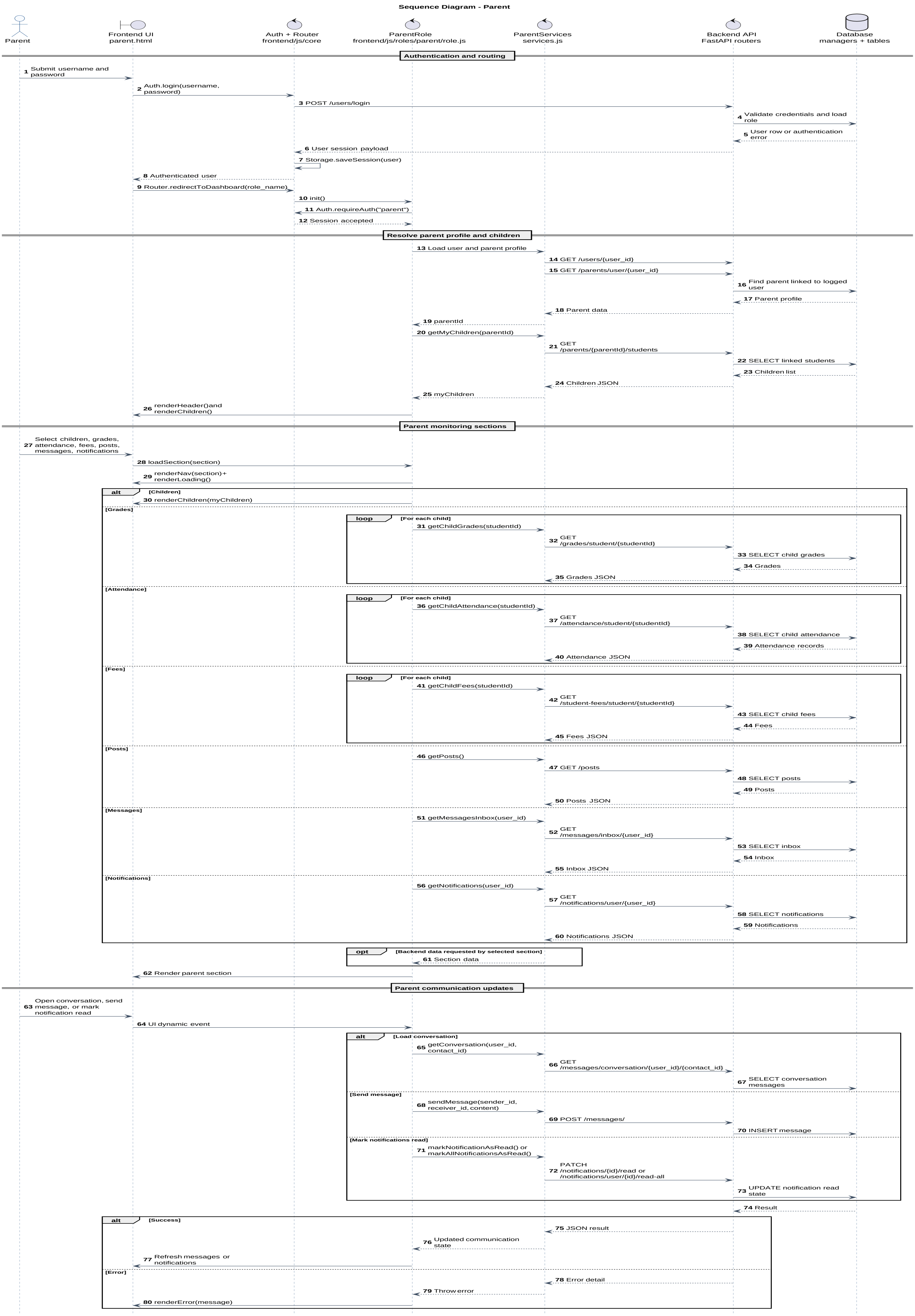
Sequence Diagram - Receptionist
Receptionist
Frontend UI receptionist.html
Auth + Router frontend/js/core
ReceptionistRole frontend/js/roles/receptionist/role.js
ReceptionistServices services.js
Backend API FastAPI routers
Database managers + tables
Authentication and routing
1 Submit username and password
2 Auth.login(username, password)
3 POST /users/login
4 Validate credentials and load role
5 User row or authentication error
6 User session payload
7 Storage.saveSession(user)
8 Authenticated user
9 Router.redirectToDashboard(role_name)
10 init()
11 Auth.requireAuth("receptionist")
12 Session accepted
Bootstrap reception desk
13 Load profile, classes, parents
14 GET /users/{user_id}
15 GET /classes/
16 GET /parents/
17 SELECT receptionist profile and reference data
18 Profile, classes, parents
19 JSON bootstrap data
20 classesList + parentsList
21 renderHeader() and loadSection("students")
Section loading
22 Select students, parents, search, finance, posts, messages, notifications
23 loadSection(section)
24 renderNav(section)+ renderLoading()
alt [Students]
25 getStudents()
26 GET /students/
[Parents]
27 getParents()
28 GET /parents/
[Search]
29 renderSearch()
[Finance]
30 renderFinanceDashboard()
[Posts]
31 getPosts()
32 GET /posts
[Messages]
33 getMessagesInbox(user_id)
34 GET /messages/inbox/{user_id}
[Notifications]
35 getNotifications(user_id)
36 GET /notifications/user/{user_id}
opt [Backend data requested by selected section]
37 SELECT requested reception data
38 Records
39 JSON response
40 Data for section
41 Render ReceptionistUI section
Register parent or student
42 Submit registration form
43 register-parent-form or register-student-form
44 createUser(userData)
45 POST /users/
46 INSERT user account
47 new user_id
48 Created user
alt [Register parent]
49 createParent(newUser.user_id)
50 POST /parents/
51 INSERT parent profile
[Register student]
52 createStudent(studentData)
53 POST /students/
54 INSERT student profile
55 Created profile
56 Creation response
57 Operation completed
58 Reload students or parents list
Student file, editing, and payment
59 View/edit student or search fees
60 Student action handler
alt [View student details]
61 getStudent(), getStudentGrades(), getStudentAttendance()
62 GET /students/{id}, /grades/student/{id}, /attendance/student/{id}
63 SELECT student file
[Edit student]
64 getStudent(), updateUser(), updateStudent()
65 GET /students/{id}, PUT /users/{id}, PUT /students/{id}
66 UPDATE user and student data
[Pay student fee]
67 getStudentFees(), processPayment()
68 GET /student-fees/student/{id}, POST /payments/
69 SELECT fee and INSERT payment
[Manage posts]
70 createPost(), updatePost(), deletePost()
71 POST/PUT/DELETE /posts/
72 Persist post operation
[Messages and notifications]
73 getConversation(), sendMessage(), markNotificationAsRead()
74 GET/POST /messages, PATCH /notifications
75 Persist communication state
76 Result
alt [Success]
77 JSON result
78 Updated data
79 Render modal, refreshed list, or success message
[Error]
80 Error detail
81 Throw error
82 Show error message


Sequence Diagram - Accountant
Accountant
Frontend UI
accountant.html
Auth + Router
frontend/js/core
AccountantRole
frontend/js/roles/accountant/role.js
AccountantServices
services.js
Backend API
FastAPI routers
Database
managers + tables
Authentication and routing
1 Submit username and password
2 Auth.login(username, password)
3 POST /users/login
4 Validate credentials and load role
5 User row or authentication error
6 User session payload
7 Storage.saveSession(user)
8 Authenticated user
9 Router.redirectToDashboard(role_name)
10 init()
11 Auth.requireAuth("accountant")
12 Session accepted
13 renderHeader(session)
Load accounting workspace
14 Select students, fees, payments, transactions, search, attendance, messages, notifications
15 loadSection(section, page)
16 renderNav(section)+ renderLoading()
alt
[Students or attendance list]
17 getStudents(page)
18 GET /students/?page={page}&limit=50
[Fees]
19 getAllFees(page)
20 GET /student-fees/?page={page}&limit=50
[Payments]
21 getPayments(page)
22 GET /payments/?page={page}&limit=50
[Transactions]
23 getTransactions(page)
24 GET /transactions/?page={page}&limit=50
[Search users]
25 getAllUsers(page) or searchUsers(keyword, page)
26 GET /users/ or /users/search
[Messages]
27 getMessagesInbox(user_id)
28 GET /messages/inbox/{user_id}
[Notifications]
29 getNotifications(user_id)
30 GET /notifications/user/{user_id}
31 SELECT paged accounting data
32 Records
33 JSON page
34 Section data
35 Render AccountantUI table
Student financial file
36 Open student finance file
37 view-student-finance-btn
38 getStudent(), getStudentFees(), getStudentPayments()
39 GET /students/{student_id}
40 GET /student-fees/student/{student_id}
41 GET /payments/student/{student_id}
42 SELECT student profile and financial history
43 Student account state
44 Modal data
45 Financial file
46 showStudentFinanceModal()
Accountant operations
47 Create fee, record payment, send notice, or fetch attendance
48 Dynamic click/submit handler
alt
[Create fee]
49 createFee(feeData)
50 POST /student-fees/
51 INSERT fee
[Record payment]
52 recordPayment(paymentData)
53 POST /payments/
54 INSERT payment
[Send finance notification]
55 sendNotification(data)
56 POST /notifications/
57 INSERT notification
[View attendance report]
58 getStudentAttendance(student_id)
59 GET /attendance/student/{student_id}
60 SELECT attendance records
[Messaging]
61 getConversation(), sendMessage()
62 GET/POST /messages
63 SELECT/INSERT messages
[Mark notifications read]
64 markNotificationAsRead() or markAllNotificationsAsRead()
65 PATCH /notifications/{id}/read or /notifications/user/{id}/read-all
66 UPDATE notification read state
67 Result
alt
[Success]
68 JSON result
69 Operation completed
70 Refresh current section or modal
[Error]
71 Error detail
72 Throw error
73 Show error message


Sequence Diagram - Teacher
Teacher
Frontend UI teacher.html
Auth + Router frontend/js/core
TeacherRole frontend/js/roles/teacher/role.js
TeacherServices services.js
Backend API FastAPI routers
Database managers + tables
Authentication and routing
1 Submit username and password
2 Auth.login(username, password)
3 POST /users/login
4 Validate credentials and load role
5 User row or authentication error
6 User session payload
7 Storage.saveSession(user)
8 Authenticated user
9 Router.redirectToDashboard(role_name)
10 init()
11 Auth.requireAuth("teacher")
12 Session accepted
Bootstrap teacher dashboard
13 Load user profile and teacher profile
14 GET /users/{user_id}
15 GET /teachers/
16 Find teacher linked to logged user
17 Teacher profile
18 Profile data
19 teacherId
20 getMyAssignments(teacherId)
21 GET /assignments/teacher/{teacherId}
22 SELECT teacher assignments
23 Assignments
24 Assignments JSON
25 myAssignments
26 renderHeader()and renderAssignments()
Load teaching sections
27 Select assignments, schedule, attendance, grades, resources, posts, messages, notifications
28 loadSection(section)
29 renderNav(section)+ renderLoading()
alt [Assignments]
30 renderAssignments(myAssignments)
[Schedule]
31 getMySchedule(teacherId)
32 GET /schedules/teacher/{teacherId}
[Attendance sheet]
33 renderAttendance(myAssignments)
34 Select class and date
35 load-attendance-btn
36 getAttendanceSheet(classId, date)
37 GET /attendance/class/{classId}/sheet?target_date={date}
[Assessments and grades]
38 renderGrades(myAssignments)
39 Select assignment or assessment
40 getAssessments(), getAssessmentGrades()
41 GET /assessments/assignment/{assignmentId}, /grades/assessment/{assessmentId}
[Resources]
42 getResources(assignmentId)
43 GET /resources/
[Posts]
44 getPosts()
45 GET /posts
[Messages]
46 getMessagesInbox(user_id)
47 GET /messages/inbox/{user_id}
[Notifications]
48 getNotifications(user_id)
49 GET /notifications/user/{user_id}
opt [Backend data requested by selected section]
50 SELECT requested teaching data
51 Records
52 JSON response
53 Section data
54 Render selected section
Teacher write operations
55 Save attendance, create assessment, save grade, upload resource, send message, mark notification
56 Dynamic event handler
alt [Save attendance]
57 saveAttendance(attendanceData)
58 POST /attendance/
59 INSERT or UPDATE attendance
[Create assessment]
60 createAssessment(assessmentData)
61 POST /assessments/
62 INSERT assessment
[Save grade]
63 saveGrade(gradeData)
64 POST /grades/
65 INSERT or UPDATE grade
[Upload resource]
66 uploadResource(formData)
67 POST /resources/upload
68 INSERT resource metadata and store file
[Messaging]
69 getConversation(), sendMessage()
70 GET/POST /messages
71 SELECT/INSERT messages
[Mark notifications read]
72 markNotificationAsRead() or markAllNotificationsAsRead()
73 PATCH /notifications/{id}/read or /notifications/user/{id}/read-all
74 UPDATE notification read state
75 Result
alt [Success]
76 JSON result
77 Operation completed
78 Confirm and refresh current view
[Error]
79 Error detail
80 Throw error
81 renderError(message)



Sequence Diagram - Student
Student
Frontend UI
student.html
Auth + Router
frontend/js/core
StudentRole
frontend/js/roles/student/role.js
StudentServices
services.js
Backend API
FastAPI routers
Database
managers + tables
Authentication and routing
1 Submit username and password
2 Auth.login(username, password)
3 POST /users/login
4 Validate credentials and load role
5 User row or authentication error
6 User session payload
7 Storage.saveSession(user)
8 Authenticated user
9 Router.redirectToDashboard(role_name)
10 init()
11 Auth.requireAuth("student")
12 Session accepted
Resolve linked student profile
13 Load user and student records
14 GET /users/{user_id}
15 GET /students/
16 Find student linked to logged user
17 Student list and user profile
18 Candidate student profile
19 studentId
20 getStudentById(studentId)
21 GET /students/{studentId}
22 SELECT full student profile
23 Student profile and class
24 Full profile
25 studentId + classId
26 renderHeader()and loadSection("schedule")
Student self-service sections
27 Select schedule, assessments, grades, attendance, resources, posts, messages, fees, notifications
28 loadSection(section)
29 renderNav(section)+ renderLoading()
alt
[Schedule]
30 getMySchedule(studentId)
31 GET /schedules/student/{studentId}
[Assessments]
32 getMyAssessments(studentId)
33 GET /assessments/student/{studentId}
[Grades]
34 getMyGrades(studentId)
35 GET /grades/student/{studentId}
[Attendance]
36 getMyAttendance(studentId)
37 GET /attendance/student/{studentId}
[Resources]
38 getResources()
39 GET /resources/
[Posts]
40 getPosts()
41 GET /posts
[Fees]
42 getMyFees(studentId)
43 GET /student-fees/student/{studentId}
[Messages]
44 getMessagesInbox(user_id)
45 GET /messages/inbox/{user_id}
[Notifications]
46 getMyNotifications(user_id)
47 GET /notifications/user/{user_id}
48 SELECT student-owned records
49 Records
50 JSON response
51 Section data
52 Render read-only student view
Student communication updates
53 Open conversation, send message, or mark notifications read
54 UI dynamic event
alt
[Load conversation]
55 getConversation(user_id, contact_id)
56 GET /messages/conversation/{user_id}/{contact_id}
57 SELECT conversation messages
[Send message]
58 sendMessage(sender_id, receiver_id, content)
59 POST /messages/
60 INSERT message
[Mark notification read]
61 markNotificationAsRead() or markAllNotificationsAsRead()
62 PATCH /notifications/{id}/read or /notifications/user/{id}/read-all
63 UPDATE notification read state
64 Result
alt
[Success]
65 JSON result
66 Updated communication state
67 Refresh messages or notifications
[Error]
68 Error detail
69 Throw error
70 renderError(message)

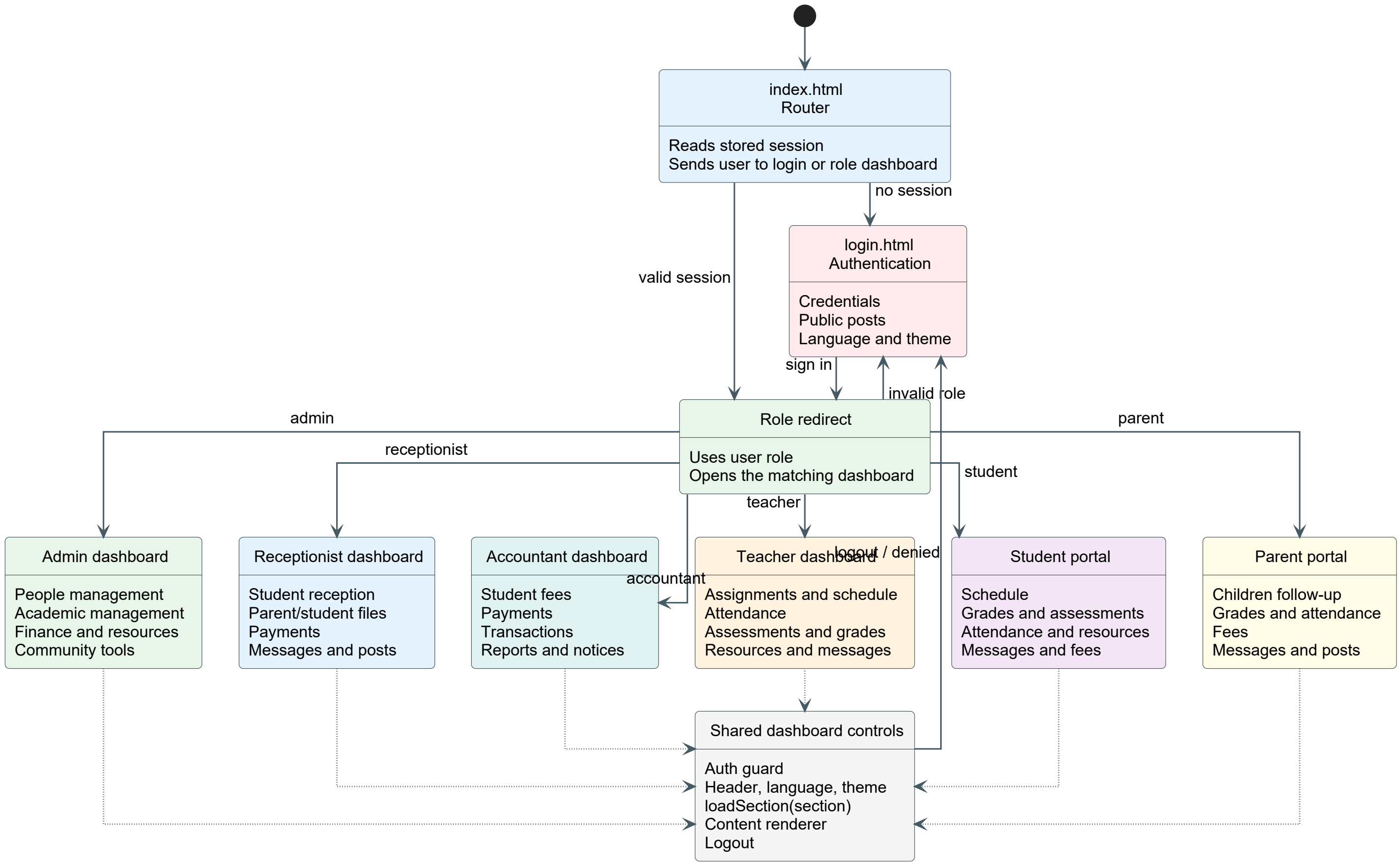

Sequence Diagram - Parent
Parent
Frontend UI
parent.html
Auth + Router
frontend/js/core
ParentRole
frontend/js/roles/parent/role.js
ParentServices
services.js
Backend API
FastAPI routers
Database
managers + tables
Authentication and routing
1 Submit username and password
2 Auth.login(username, password)
3 POST /users/login
4 Validate credentials and load role
5 User row or authentication error
6 User session payload
7 Storage.saveSession(user)
8 Authenticated user
9 Router.redirectToDashboard(role_name)
10 init()
11 Auth.requireAuth("parent")
12 Session accepted
Resolve parent profile and children
13 Load user and parent profile
14 GET /users/{user_id}
15 GET /parents/user/{user_id}
16 Find parent linked to logged user
17 Parent profile
18 Parent data
19 parentId
20 getMyChildren(parentId)
21 GET /parents/{parentId}/students
22 SELECT linked students
23 Children list
24 Children JSON
25 myChildren
26 renderHeader()and renderChildren()
Parent monitoring sections
27 Select children, grades, attendance, fees, posts, messages, notifications
28 loadSection(section)
29 renderNav(section)+ renderLoading()
alt
[Children]
30 renderChildren(myChildren)
[Grades]
loop
[For each child]
31 getChildGrades(studentId)
32 GET /grades/student/{studentId}
33 SELECT child grades
34 Grades
35 Grades JSON
[Attendance]
loop
[For each child]
36 getChildAttendance(studentId)
37 GET /attendance/student/{studentId}
38 SELECT child attendance
39 Attendance records
40 Attendance JSON
[Fees]
loop
[For each child]
41 getChildFees(studentId)
42 GET /student-fees/student/{studentId}
43 SELECT child fees
44 Fees
45 Fees JSON
[Posts]
46 getPosts()
47 GET /posts
48 SELECT posts
49 Posts
50 Posts JSON
[Messages]
51 getMessagesInbox(user_id)
52 GET /messages/inbox/{user_id}
53 SELECT inbox
54 Inbox
55 Inbox JSON
[Notifications]
56 getNotifications(user_id)
57 GET /notifications/user/{user_id}
58 SELECT notifications
59 Notifications
60 Notifications JSON
opt
[Backend data requested by selected section]
61 Section data
62 Render parent section
Parent communication updates
63 Open conversation, send message, or mark notification read
64 UI dynamic event
alt
[Load conversation]
65 getConversation(user_id, contact_id)
66 GET /messages/conversation/{user_id}/{contact_id}
67 SELECT conversation messages
[Send message]
68 sendMessage(sender_id, receiver_id, content)
69 POST /messages/
70 INSERT message
[Mark notifications read]
71 markNotificationAsRead() or markAllNotificationsAsRead()
72 PATCH /notifications/{id}/read or /notifications/user/{id}/read-all
73 UPDATE notification read state
74 Result
alt
[Success]
75 JSON result
76 Updated communication state
77 Refresh messages or notifications
[Error]
78 Error detail
79 Throw error
80 renderError(message)

User Interface: Navigation Diagram

The navigation diagram documents how users move from the entry point and authentication page into their role-specific dashboards. It also shows the shared frontend controls and the major section families exposed by each role.

Navigation Flow

Entry: users start from the index/login flow, where authentication determines the target dashboard.
Dashboards: Admin, Receptionist, Accountant, Teacher, Student, and Parent each have a dedicated page and role controller.
Sections: navigation buttons call the active role controller, which loads the selected section and renders the corresponding UI.
Shared controls: authentication checks, language switching, theme preferences, logout, messaging, and notifications are reused across dashboards.


index.html
Router
Reads stored session
Sends user to login or role dashboard
no session
valid session
login.html
Authentication
Credentials
Public posts
Language and theme
sign in
invalid role
Role redirect
Uses user role
Opens the matching dashboard
admin
receptionist
parent
student
teacher
accountant
logout / denied
Admin dashboard
People management
Academic management
Finance and resources
Community tools
Receptionist dashboard
Student reception
Parent/student files
Payments
Messages and posts
Accountant dashboard
Student fees
Payments
Transactions
Reports and notices
Teacher dashboard
Assignments and schedule
Attendance
Assessments and grades
Resources and messages
Student portal
Schedule
Grades and assessments
Attendance and resources
Messages and fees
Parent portal
Children follow-up
Grades and attendance
Fees
Messages and posts
Shared dashboard controls
Auth guard
Header, language, theme
loadSection(section)
Content renderer
Logout

Technology Stack, Deployment, and Repository

This section lists the main technologies, deployment notes, and repository reference.

Frontend Layer

The frontend is a static web application built with **HTML5**, **CSS3**, and **vanilla JavaScript**. It uses shared core modules for routing, authentication, local storage, API calls, preferences, and internationalization. Each role has its own page, controller, service layer, and UI renderer.

Backend API Layer

The backend is built with **Python** and **FastAPI**. The API is organized into routers for users, parents, students, teachers, classes, enrollments, schedules, attendance, assessments, grades, resources, fees, payments, transactions, notifications, messages, posts, and programs. **Uvicorn** is used as the ASGI server and **Pydantic** supports request/response validation.

Database and Persistence

The persistence layer uses a relational database accessed through **mysql-connector-python** and **SQLAlchemy** utilities. Database managers separate identity, academic, learning, finance, and communication responsibilities.

Documentation and UML Report

UML diagrams are exported as **SVG** assets and embedded into the PDF report with **ReportLab** and **svglib**. The report covers use case diagrams, the class diagram, sequence diagrams, and the navigation diagram.

Hosting and Cloud Access

Port forwarding was configured for the frontend and backend to enable remote testing. Cloud services supported the hosted demonstration.

Artificial Intelligence Assistance

AI-assisted tools supported analysis, code review, UML/report refinement, and documentation.

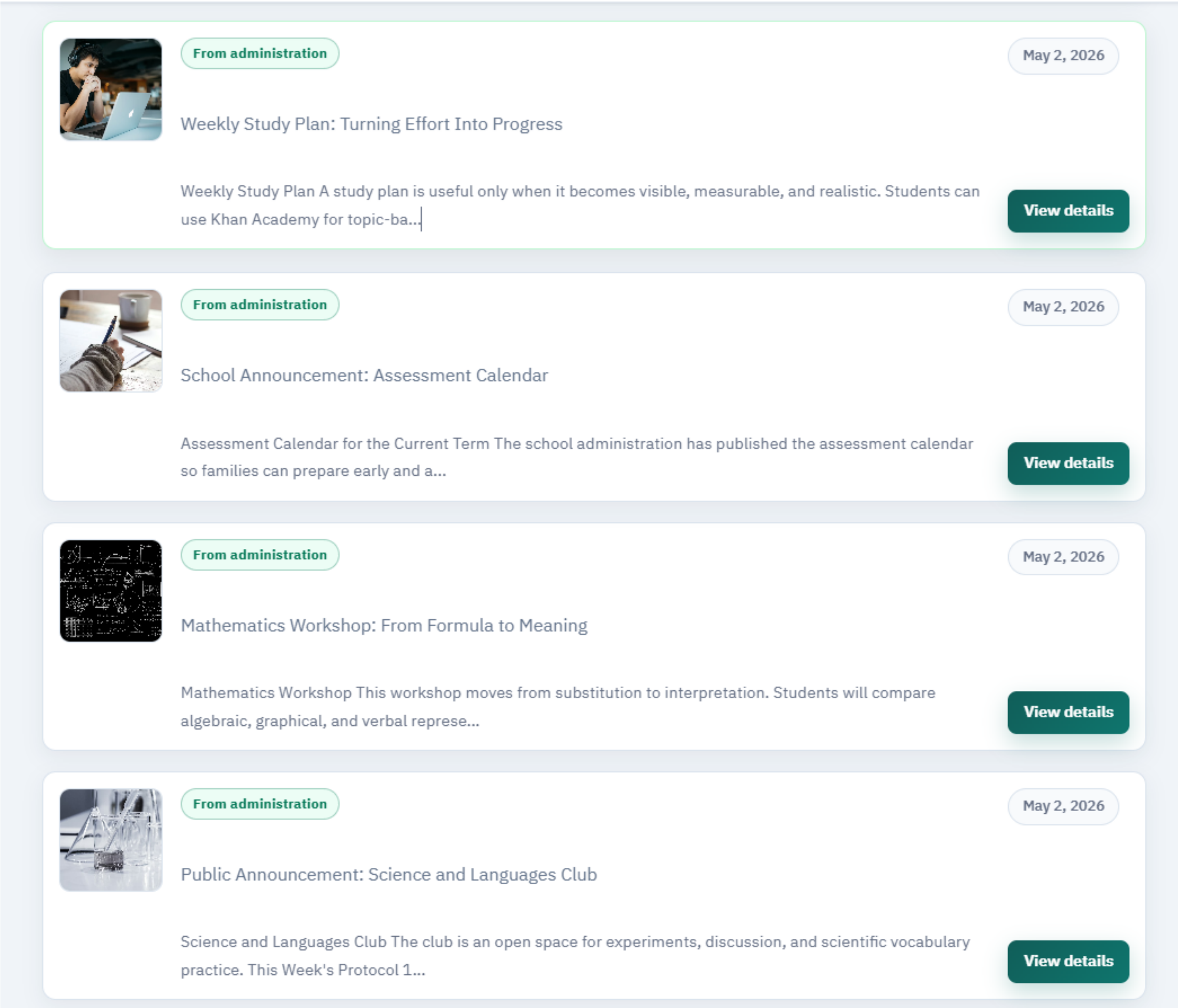
Project Repository

The project source code is available on GitHub:
https://github.com/MohammedBoure/SMS-FE-BE

Application Interface Showcase

The following screenshots present the visual side of the implemented application. Each image represents one concrete interface from the role-based dashboards.

Posts Board



Post Details

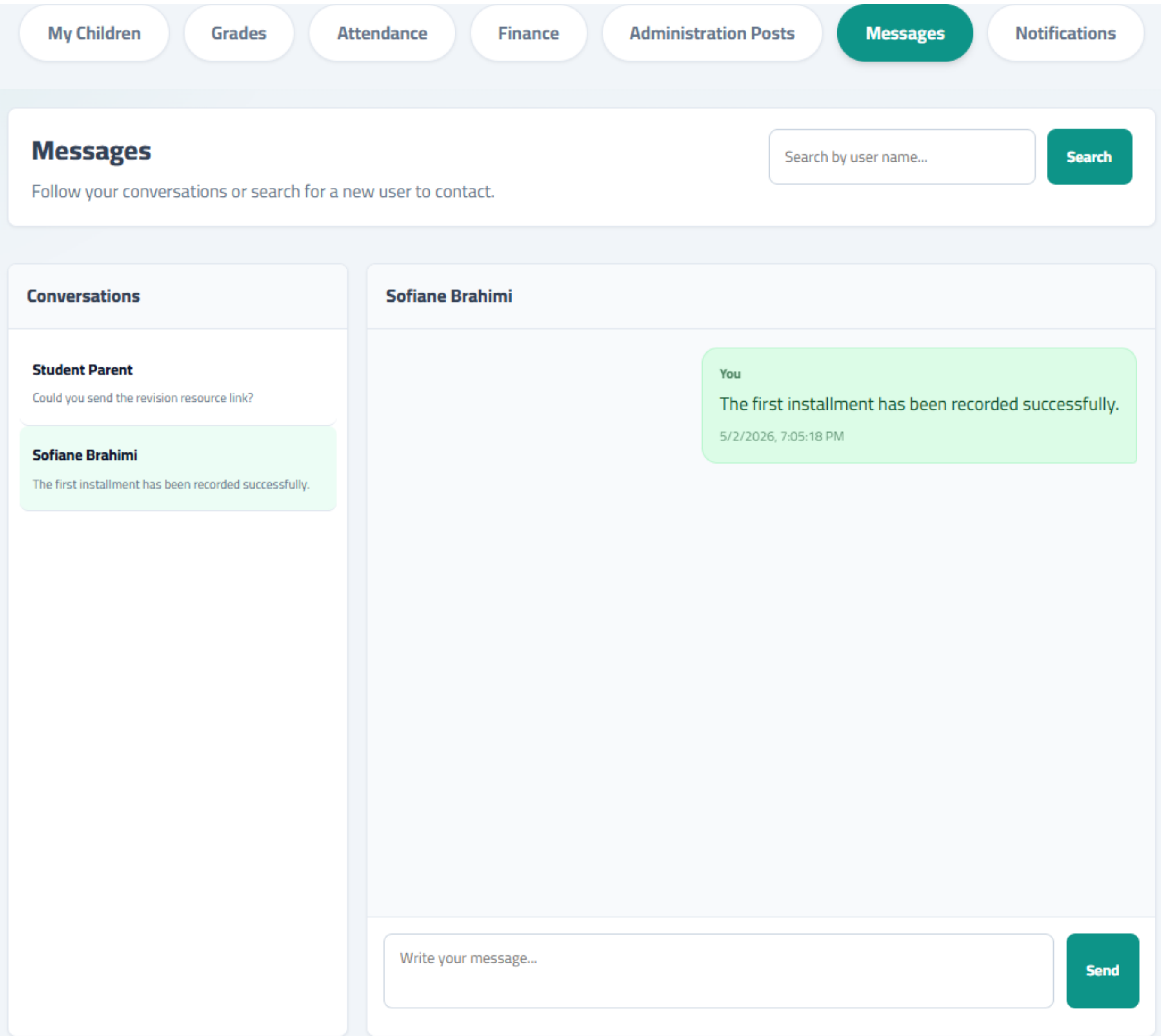
Students will work in teams to plant, observe, and document the growth of classroom plants. The project connects science, responsibility, and writing.

Role	Evidence to Submit
Observer	Height chart
Photographer	One image per week
Writer	80-word reflection

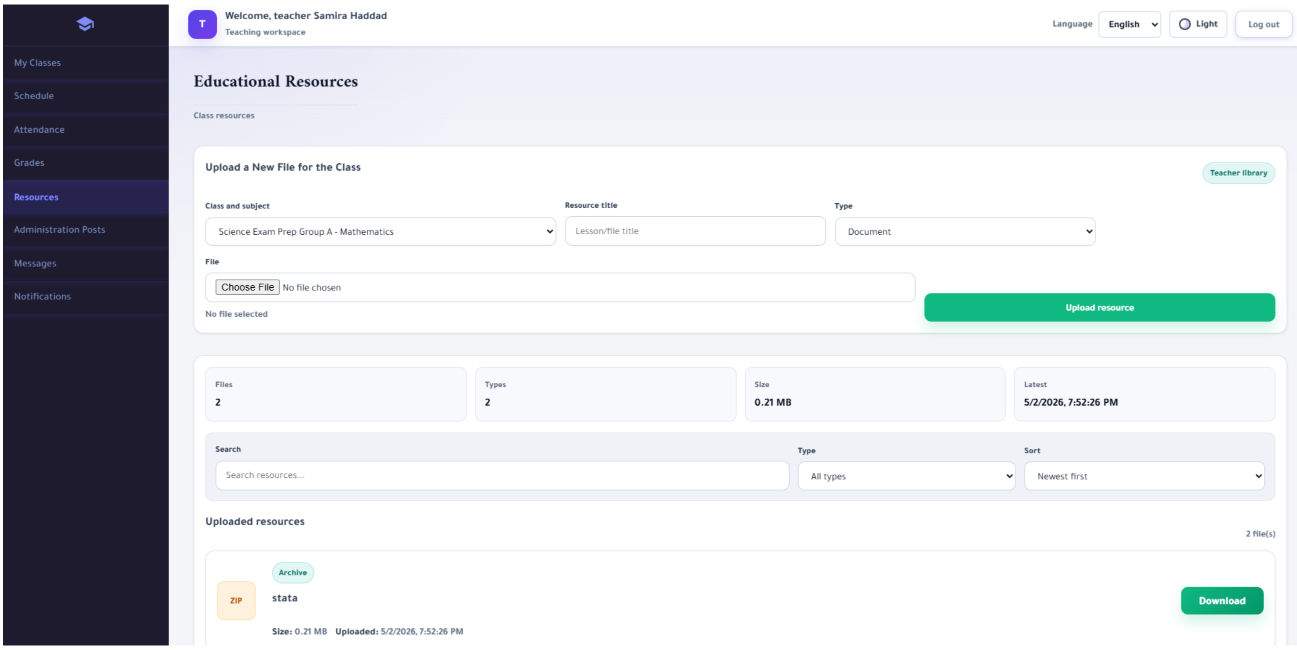
Plant growth can be modeled with a simple logistic curve:

$$h(t) = \frac{K}{1 + Ae^{-rt}}$$

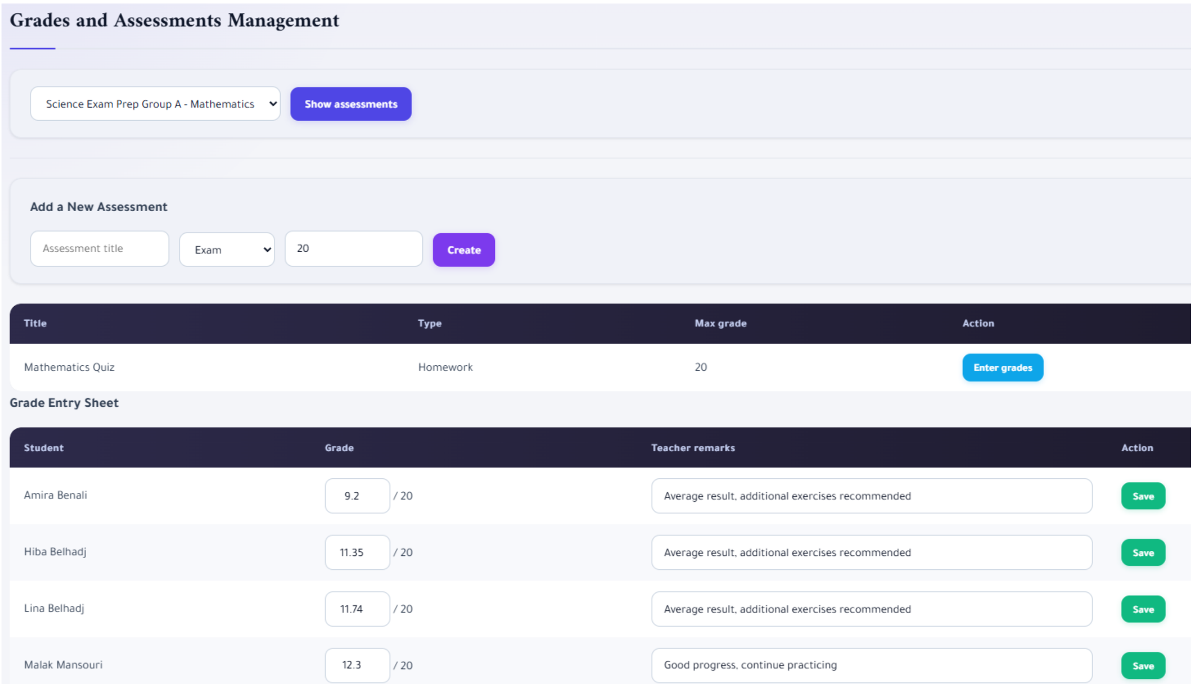
Parent Messages



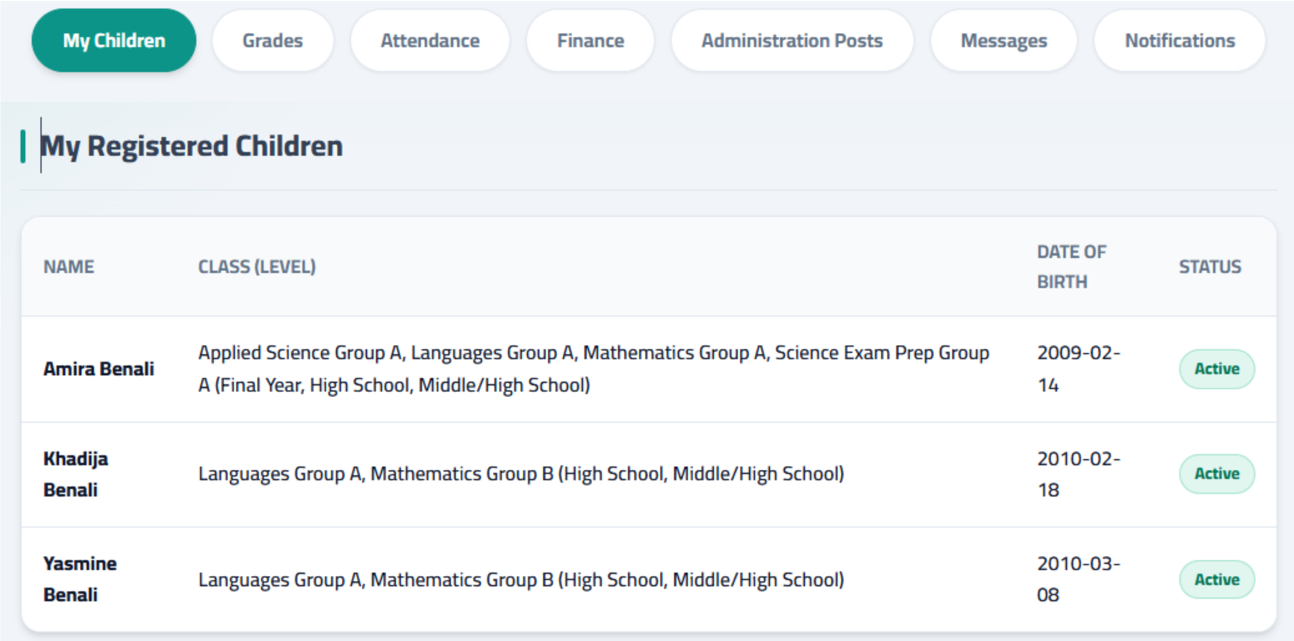
Teacher Resources



Teacher Grades



Parent - My Children



My Registered Children

NAME	CLASS (LEVEL)	DATE OF BIRTH	STATUS
Amira Benali	Applied Science Group A, Languages Group A, Mathematics Group A, Science Exam Prep Group A (Final Year, High School, Middle/High School)	2009-02-14	Active
Khadija Benali	Languages Group A, Mathematics Group B (High School, Middle/High School)	2010-02-18	Active
Yasmine Benali	Languages Group A, Mathematics Group B (High School, Middle/High School)	2010-03-08	Active

Conclusion

Project Summary

The UML documentation gives a structured view of the system: role responsibilities, backend structure, frontend-to-backend interactions, and navigation flow.

Design Value

The diagrams make the project easier to review and extend. The data structure was first studied from the Al Abakera Excel reference, then modeled into UML, backend APIs, frontend dashboards, and a hosted web setup.

Repository and Tooling Note

The project repository is available at **https://github.com/MohammedBoure/SMS-FE-BE**. AI-assisted tools supported analysis, documentation refinement, and UML/report quality improvements.